UVA CS 4774: Machine Learning

Part 2: Quiz + QA

Related:

- Part 1 [HERE]
- Course Announcement Page [HERE]

Dr. Yanjun Qi

University of Virginia

Department Of Computer Science

09/30

09/30/2025 Assignments

- HW2 is due this coming Sunday midnight!
  - Using HW1 code pieces as components;
  - If you struggle with HW1, please contact TA @Haochen ASAP
- HW1 grading is work-in-progress,
  - Grades will be released by next Tuesday class time
  - We posted the guide from TA in Canvas
- Course vote:
  - New Survey that needs your vote on
    - 1. back to lecture in-person twice a week?
    - 2. If not, best way to use the in-person session:
      - Quiz to continue
      - + Project discussions
        - Interested in Shark Tank alike setup? idea screening, pitch talk, demo …

Project Process

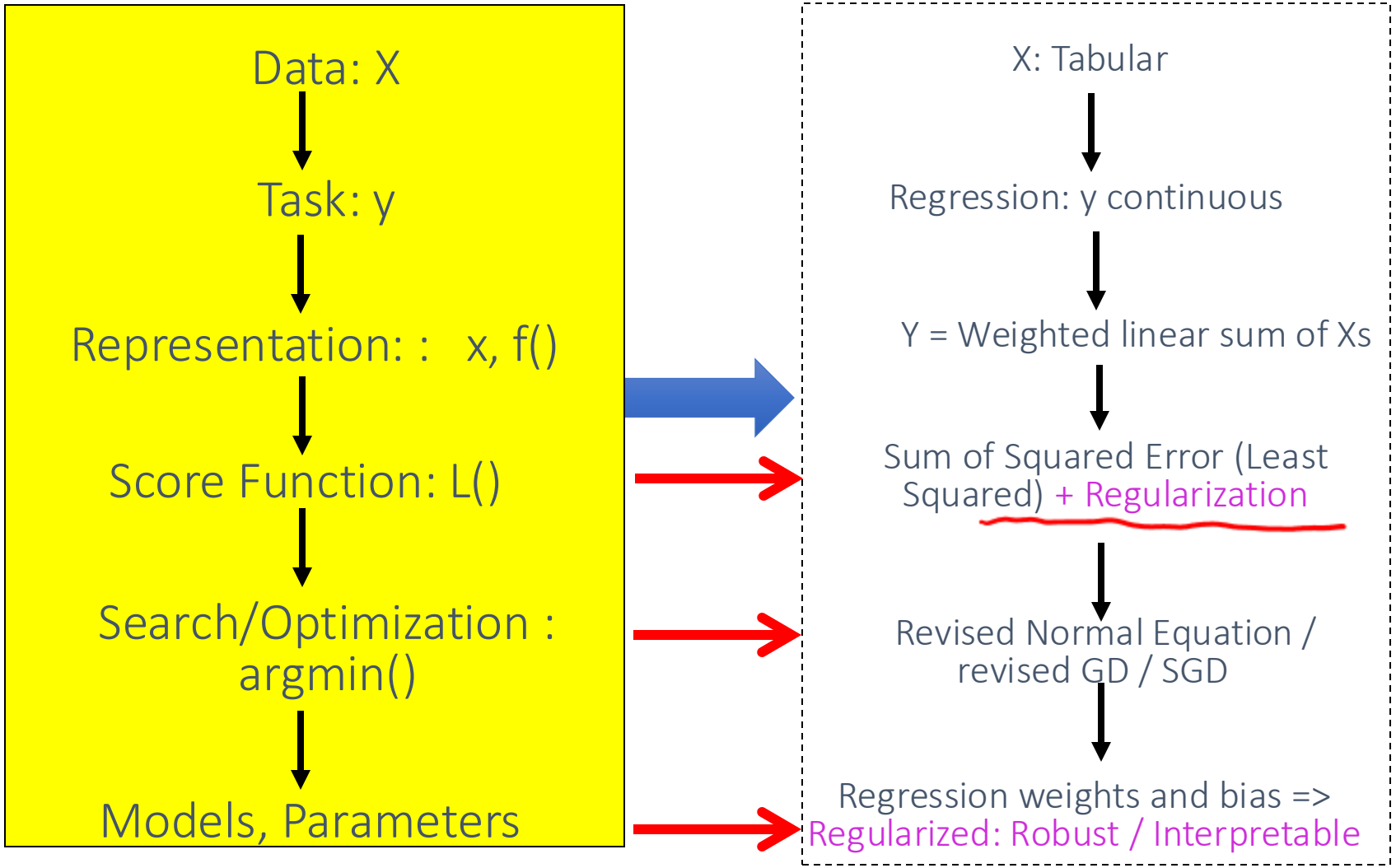
- Format:
  - Team, individual …
  - Shark Tank alike Screening ? – https://en.wikipedia.org/wiki/Shark_Tank
  - Next week – Idea collection

- Final deliverables:
  - (1) Code (Github PR to course project repo)
  - (2) Poster presentation class wide (Date: TBD)
  - (3) Video Demo (TBD)

09/30/2025 Roadmap

- TA to go over HW1
- One UVA ML club to introduce their setups and projects
- Q5
- Review Q4
- Review QA for L5-L7

L7: Regularized multivariate linear regression



L7: Regularized multivariate linear regression

We aim to make our trained model

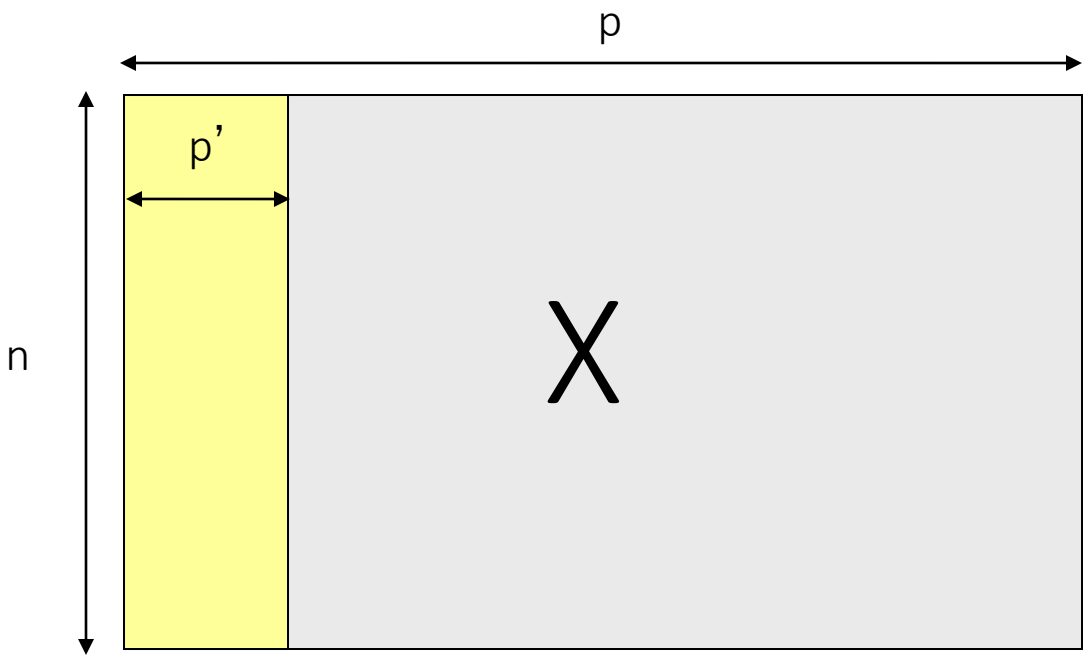
- 1. Generalize Well

- 2. Computational Scalable and Efficient

- 3. Trustworthy: Robust / Interpretable
  - Especially for some domains, this is about trust!

Large p, small n: How?

$p \rightarrow p' \Rightarrow$ { easy to understand



Regularized multivariate linear regression

- Model: $\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p$
  - LR estimation: $\arg\min \sum \left(Y - \hat{Y} \right)^2$
  - LASSO estimation: $\arg\min \sum_{i=1}^{n} \left(Y - \hat{Y} \right)^2 + \lambda \sum_{j=1}^{p} |\beta_j|$
  - Ridge regression estimation: $\arg\min \sum_{i=1}^{n} \left(Y - \hat{Y} \right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$

9/54

Error on data + Regularization

Ridge Regression / L2 Regularized Regression

$$\beta^* = \left(X^T X\right)^{-1} X^T \vec{y}$$

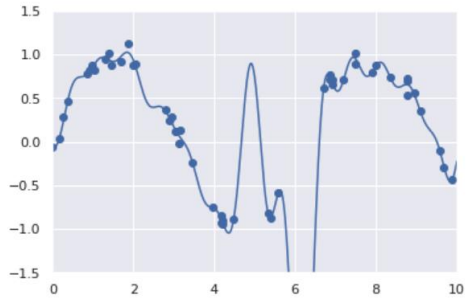
- If not invertible, a classical solution is to add a small positive element to diagonal

$$\beta^* = \left(X^T X + \lambda I\right)^{-1} X^T \vec{y}$$

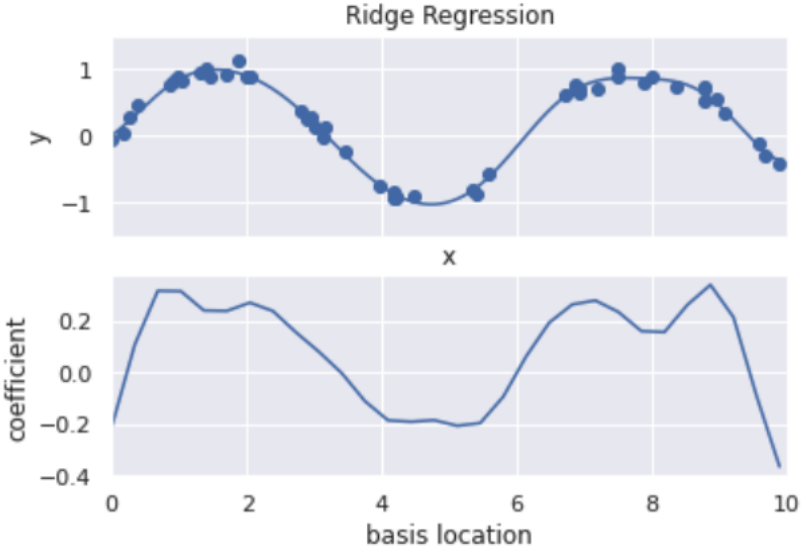
By convention, the bias/intercept term is typically not regularized. Here we assume data has been centered … therefore no bias term

Overfitting: Can be Handled by Regularization

A regularizer is an additional criteria to the loss function to make sure that we don't overfit. It's called a regularizer since it tries to keep the parameters more normal/regular




```
from sklearn.linear_model import Ridge
model = make_pipeline(GaussianFeatures(30), Ridge(alpha=0.1))
basis_plot(model, title='Ridge Regression')
```



WHY and How to Select λ?

- 1. Generalization ability
  ➔ k-folds CV to decide

- 2. Control the bias and Variance of the model (details in future lectures)

L2: Squared weights penalizes large values more

L1: Sum of weights will penalize small values more

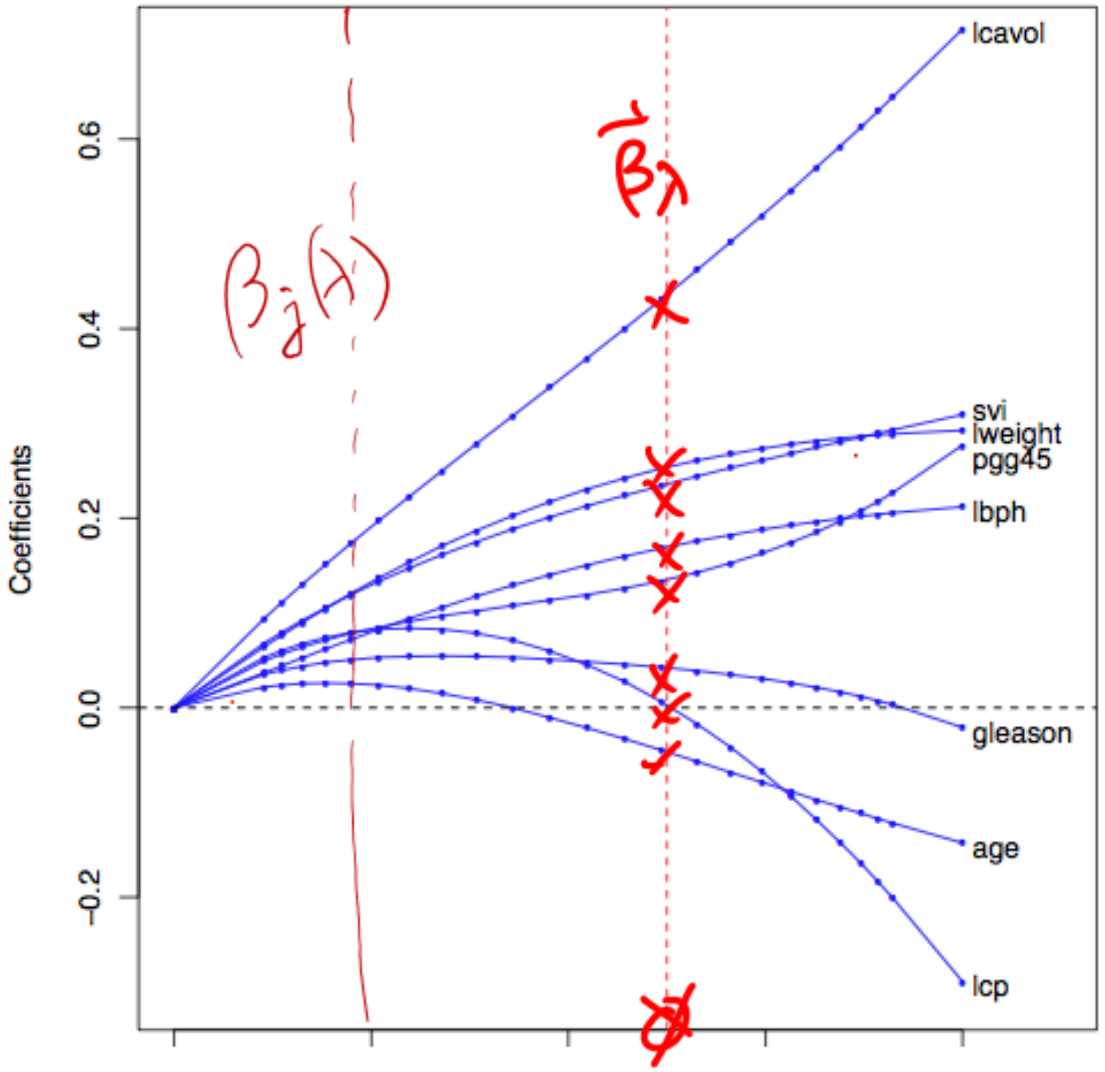
$$\square \ \beta_j^2$$
$$j$$

$$\square \ |\beta_j|$$
$$j$$

Regularization path of a Ridge Regression

when $X^T X = I \Rightarrow \frac{1}{1+\lambda}\beta_{OLS}$

Weight Decay



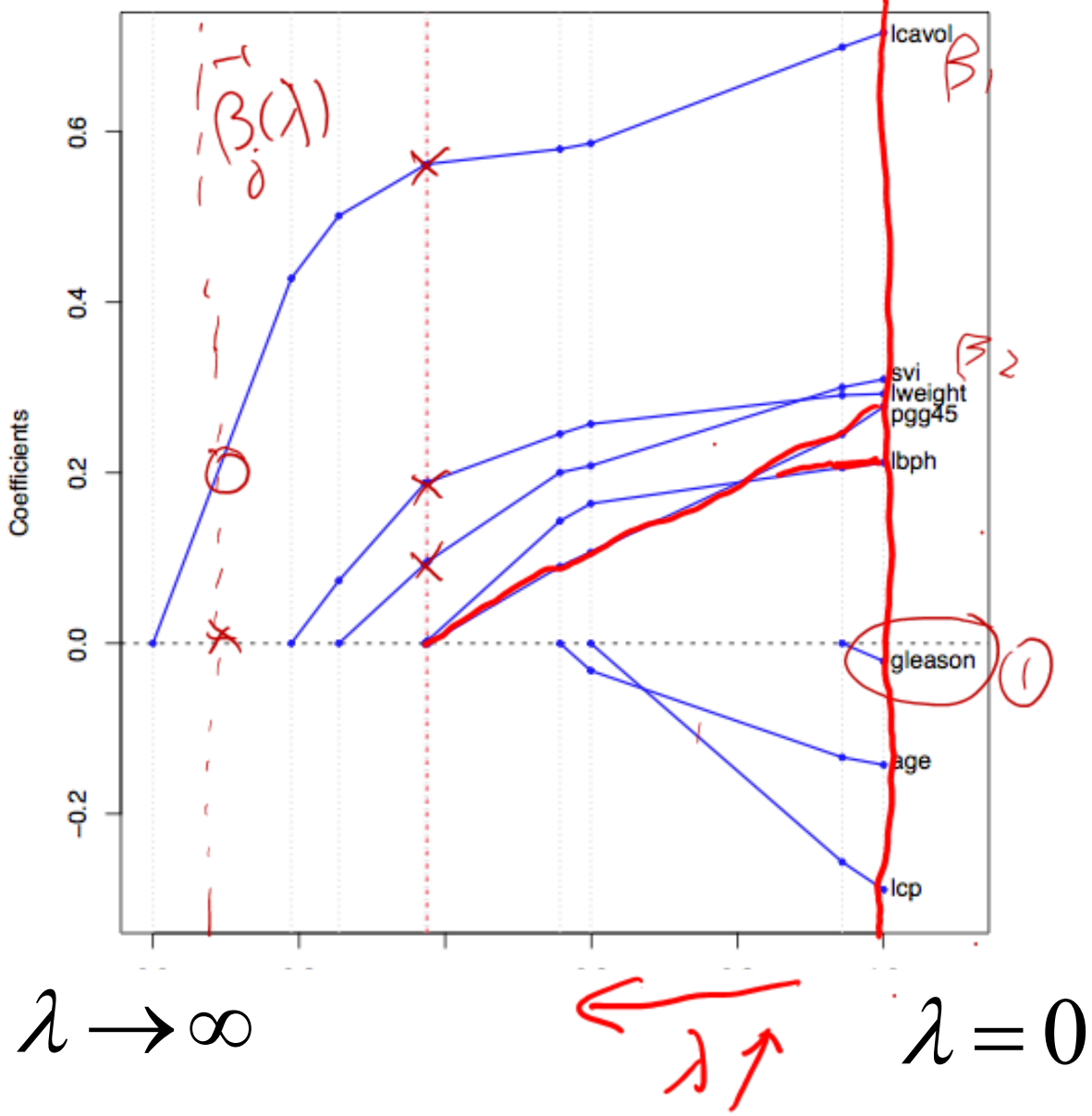
WHY and How to Select λ?

$\lambda \to \infty$ $\qquad$ $\lambda = 0$

An example with 8 features

Regularization path of a Lasso Regression

when varying λ, how β_j varies.

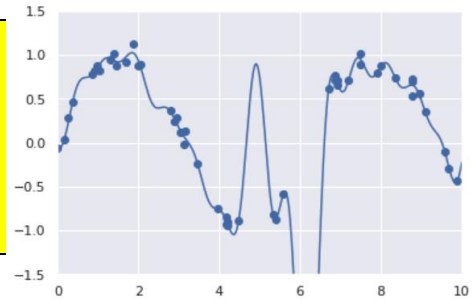


$p = 8$

An example with 8 features

Overfitting: Can be Handled by Regularization

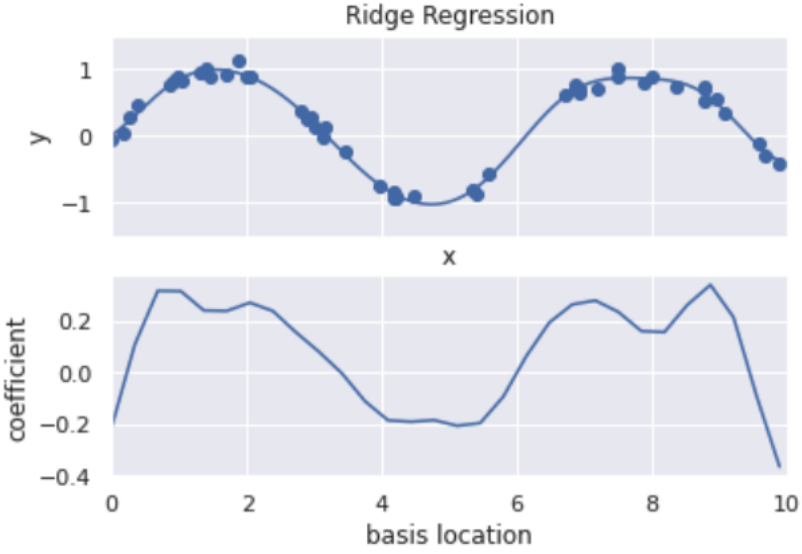
A regularizer is an additional criteria to the loss function to make sure that we don't overfit. It's called a regularizer since it tries to keep the parameters more normal/regular



code-run:

https://github.com/qiyanjun/2025Fall-UVA-CS-MachineLearningDeep/blob/main/notebook/L7_regularizedRegression_06_Linear_Regression.ipynb

```
from sklearn.linear_model import Ridge
model = make_pipeline(GaussianFeatures(30), Ridge(alpha=0.1))
basis_plot(model, title='Ridge Regression')
```



(Extra) Lasso (least absolute shrinkage and selection operator) / Squared Loss+L1

- The lasso is a shrinkage method like ridge, but acts in a nonlinear manner on the outcome y.
- The lasso is defined by

$$\sum_{i=1}^{n}(y_i - x_i^T\beta)^2$$

$$\hat{\beta}^{lasso} = \operatorname{argmin}(y - X\ \beta)^T(y - X\ \beta)$$

$$\text{subject to} \sum |\beta_j| \le s$$

L1 norm

By convention, the bias/intercept term is typically not regularized.
Here we assume data has been centered ... therefore no bias term

Lasso Estimator

Ridge Regression

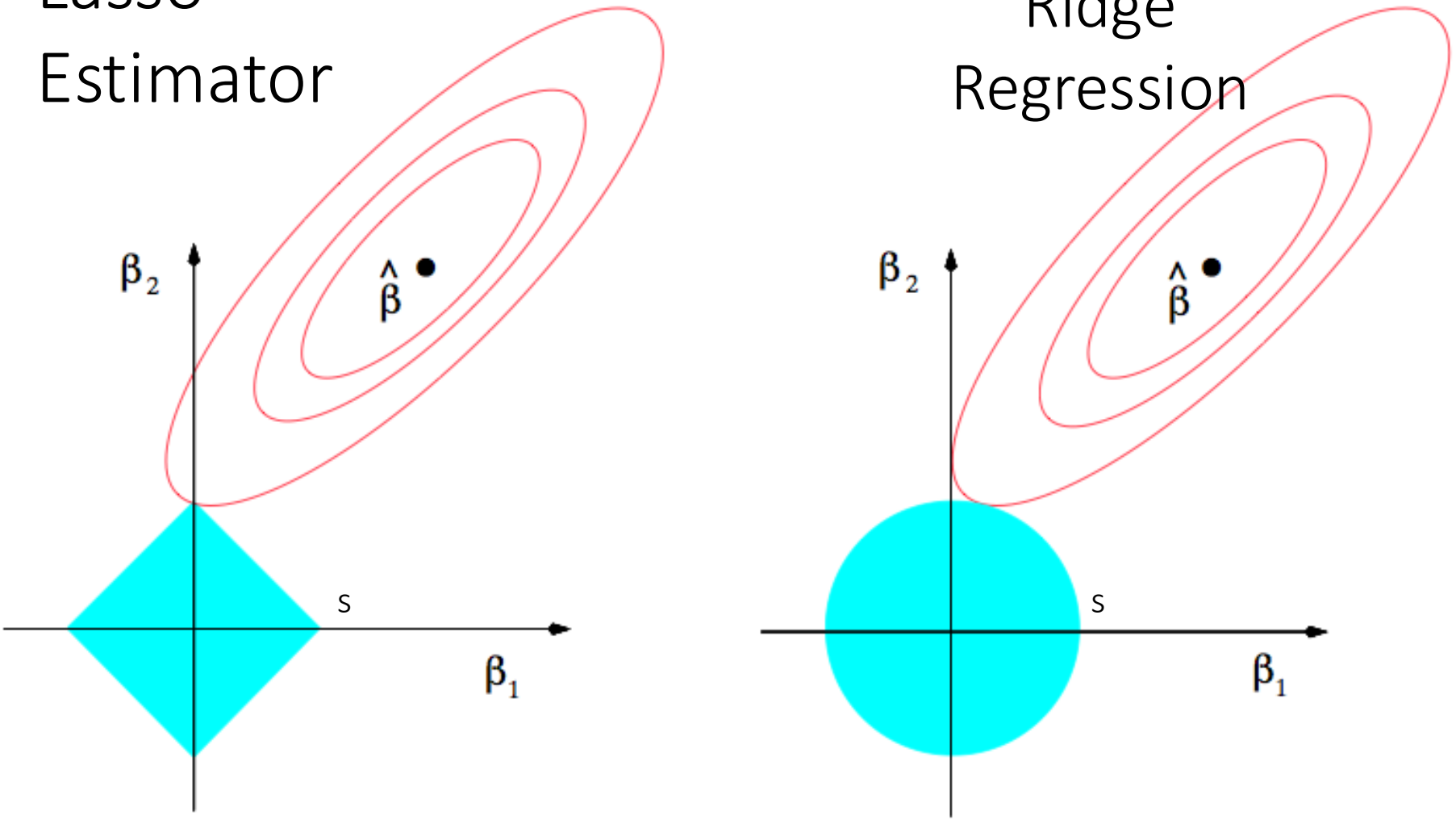
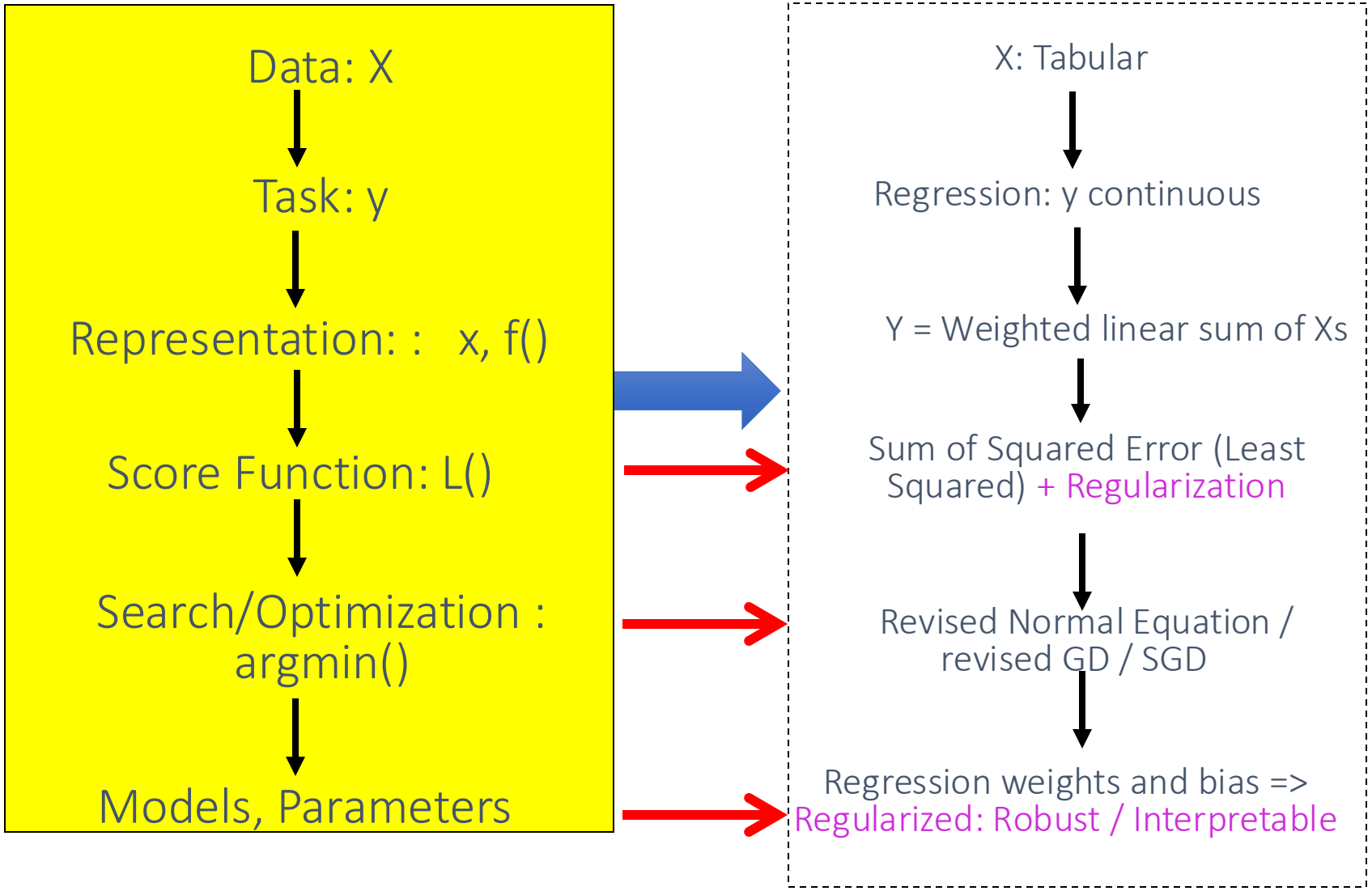


FIGURE 3.11. *Estimation picture for the lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions* $|\beta_1| + |\beta_2| \leq t$ *and* $\beta_1^2 + \beta_2^2 \leq t^2$*, respectively, while the red ellipses are the contours of the least squares error function.*

$$\min J(\beta) = \sum_{i=1}^{n}\left(Y - \hat{Y}\right)^2 + \lambda\left(\sum_{j=1}^{p}\beta_j^q\right)^{1/q}$$

Today: Regularized multivariate linear regression



More: A family of shrinkage estimators

$$\beta = \arg\min_{\beta} \sum_{i=1}^{N} (y_i - x_i^T \beta)^2$$

$$\text{subject to} \sum_j |\beta_j|^q \le s$$

- for q >=0, contours of constant value of $\sum_j |\beta_j|^q$ are shown for the case of two inputs.

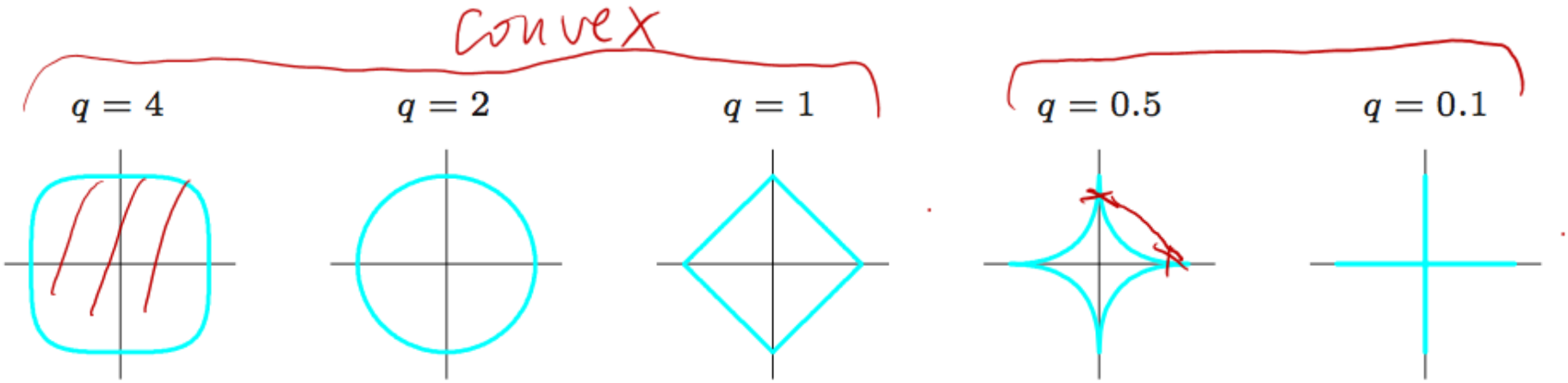
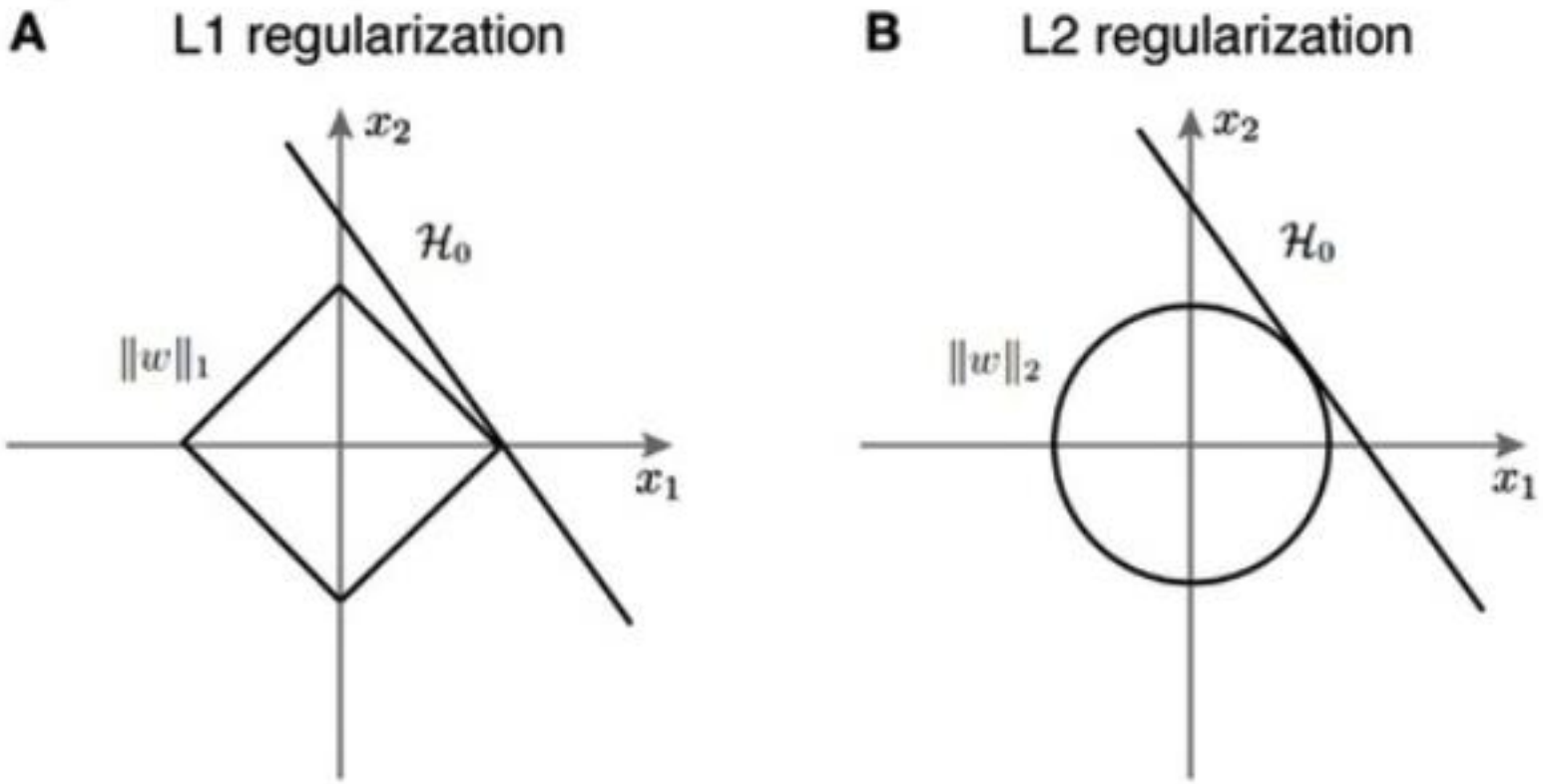


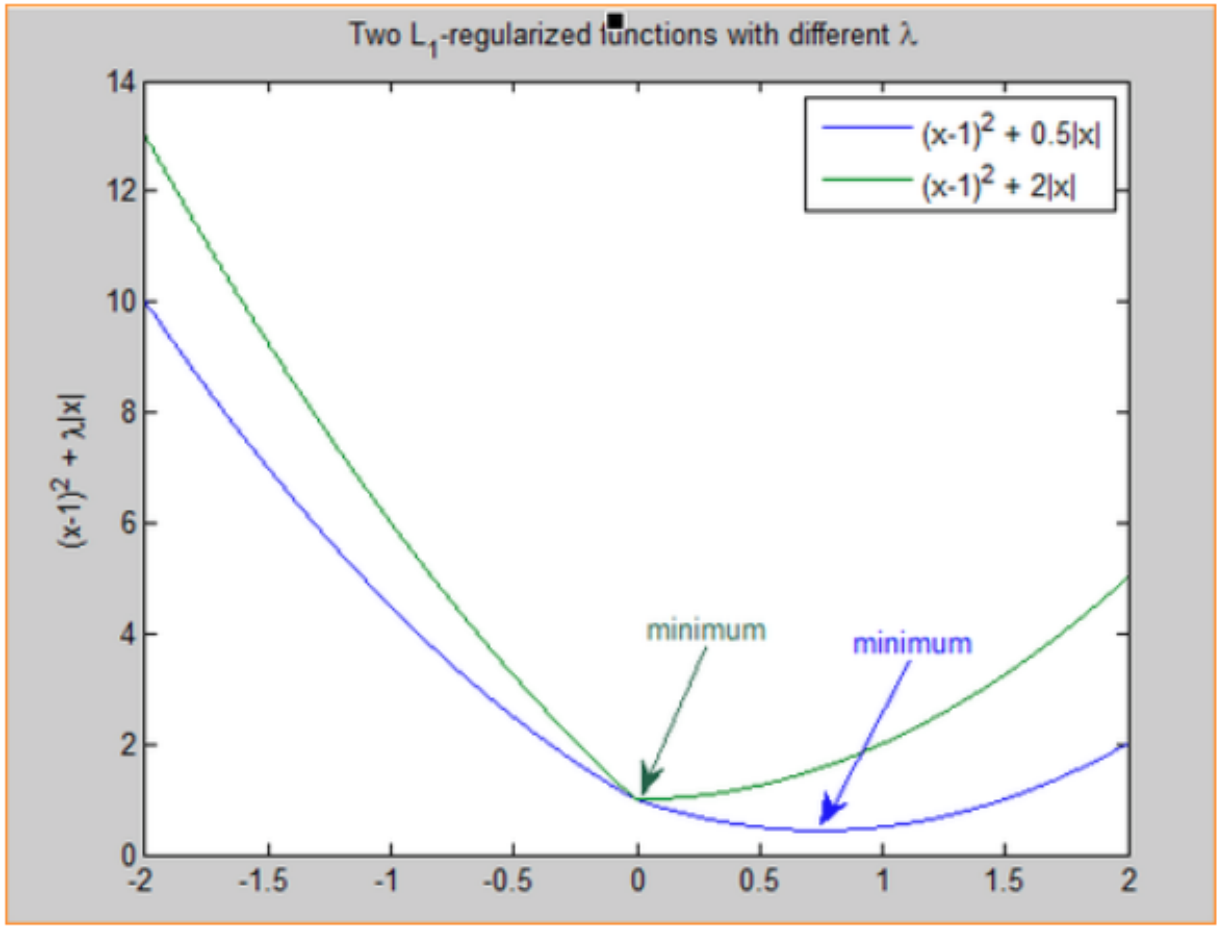
FIGURE 3.12. *Contours of constant value of* $\sum_j |\beta_j|^q$ *for given values of* q.



due to the nature of L_1 norm, the viable solutions are limited to corners, which are on a few axis only - in the above case x1. Value of x2 = 0. This means that the solution has eliminated the role of x2, leading to sparsity

L_1-regularized loss function $F(x) = f(x) + \lambda\|x\|_1$ is non-smooth. It's not differentiable at 0. Optimization theory says that the optimum of a function is either the point with 0-derivative or one of the irregularities (corners, kinks, etc.). So, it's possible that the optimal point of F is 0 even if 0 isn't the stationary point of f. In fact, it would be 0 if λ is large enough (stronger regularization effect). Below is a graphical illustration.

http://www.quora.com/What-is-the-difference-between-L1-and-L2-regularization



In mathematics, particularly in calculus, a stationary point or critical point of a differentiable function of one variable is a point of the domain of the function where the derivative is zero (equivalently, the slope of the graph at that point is zero).

Coordinate descent based Learning of Lasso

1. Initialize β
2. Repeat until converged
3. For $j = 1, 2, \ldots, p$ do

$$a_j = 2 \sum_{i=1}^{n} x_{ij}^2$$

$$c_j = 2 \sum_{i=1}^{n} x_{ij} \left(y_i - x_i^T \beta + x_{ij} \beta_j \right)$$

if $c_j < -\lambda$

$$\beta_j = (c_j + \lambda) / a_j$$

else if, $c_j > \lambda$

$$\beta_j = (c_j - \lambda) / a_j$$

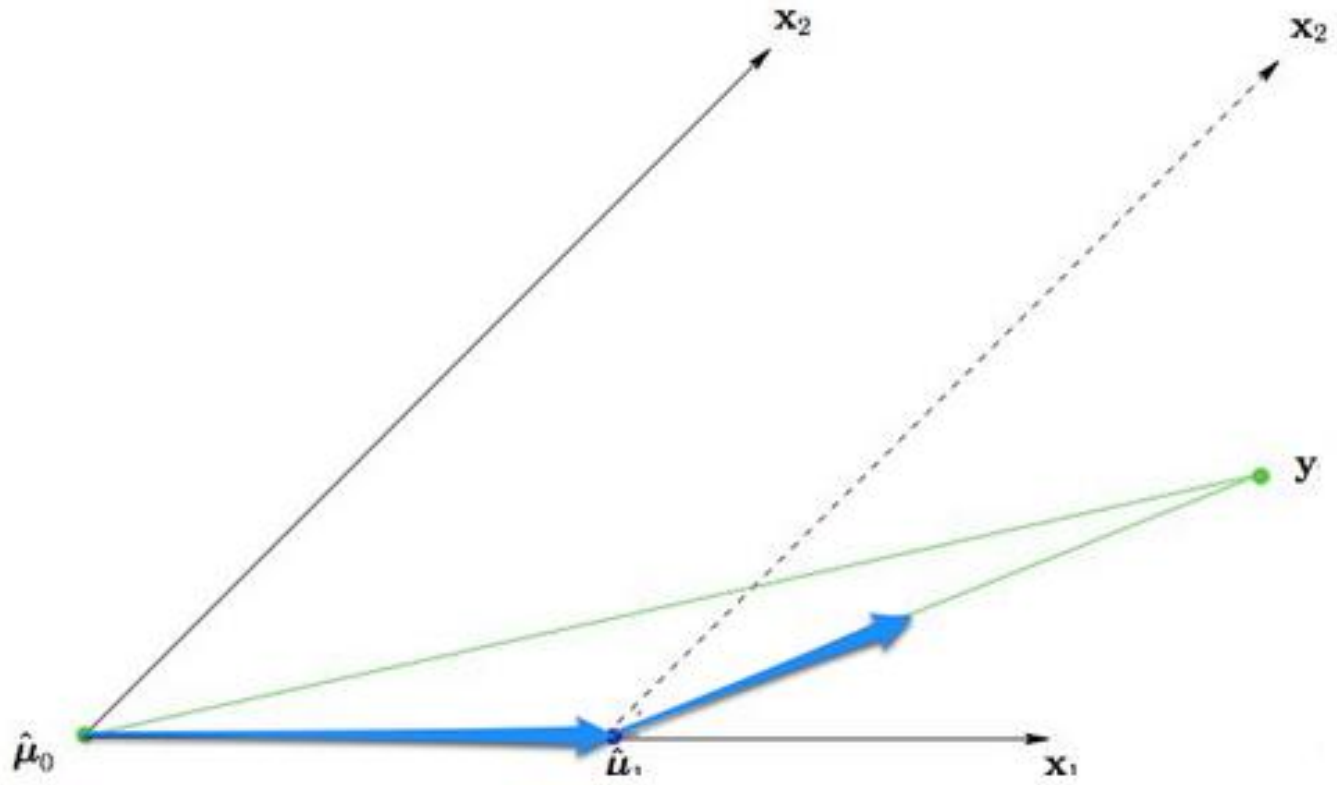
else, soft-thresholding

$$\beta_j = 0$$

Coordinate descent (WIKI)➔ one does line search along one coordinate direction at the current point in each iteration.

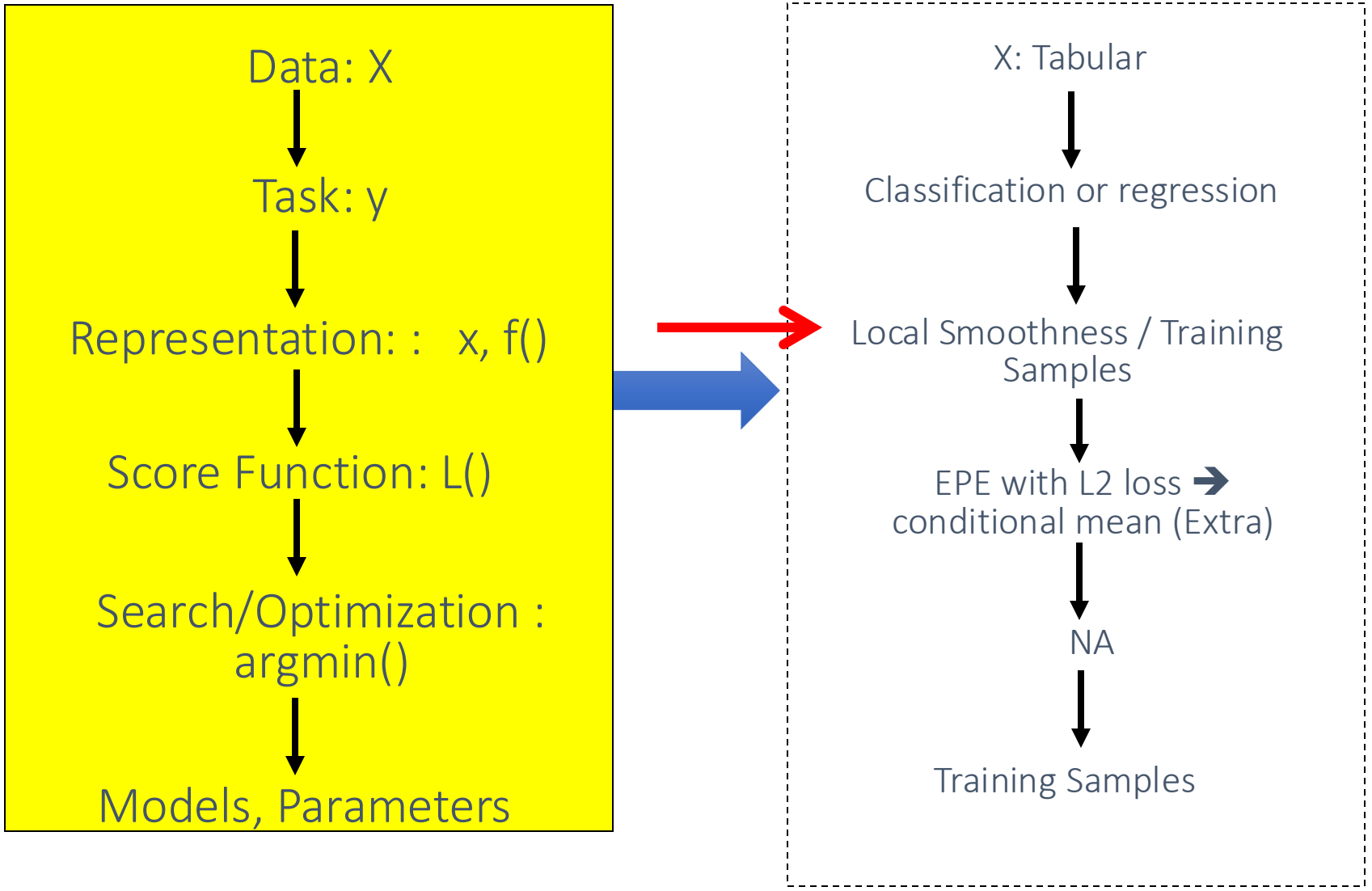
One uses different coordinate directions cyclically throughout the procedure.

Least Angle Regression (LARS) (State-of-the-art LASSO solver)



http://statweb.stanford.edu/~tibs/ftp/lars.pdf

L8: **K-nearest-neighbor(regressor or classifier)**

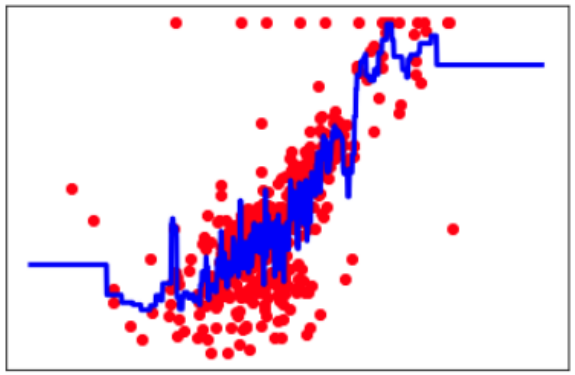


Code run: https://github.com/qiyanjun/2025Fall-UVA-CS-MachineLearningDeep/blob/main/notebook/L8_Knearest.ipynb

```
[42] # import regressor
     from sklearn.neighbors import KNeighborsRegressor
     # instantiate with K=5
     knn = KNeighborsRegressor(n_neighbors=5)
     # fit with data
     knn.fit(X, y)
```

```
KNeighborsRegressor(algorithm='auto', leaf_size=3
                    metric_params=None, n_jobs=No
                    weights='uniform')
```

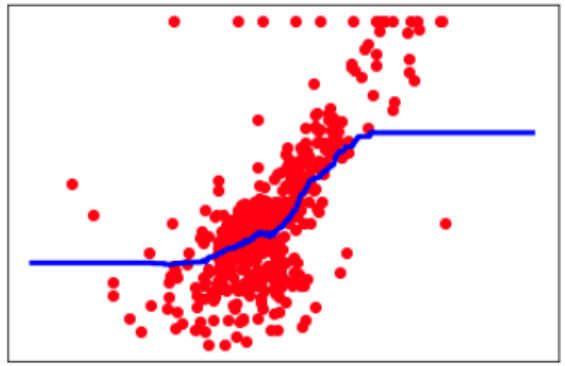
```
###
Xfit = np.linspace(3, 10, 1000).reshape(-1, 1)
yfit =knn.predict(Xfit)

# Plot outputs
plt.scatter(X, y,  color='red')
plt.plot(Xfit, yfit, color='blue', linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



```
[44] # import regressor
     from sklearn.neighbors import KNeighborsRegressor
     # instantiate with K=5
     knn = KNeighborsRegressor(n_neighbors=100)
     # fit with data
     knn.fit(X, y)
```

```
KNeighborsRegressor(algorithm='auto', leaf_size=3(
                    metric_params=None, n_jobs=Nor
                    weights='uniform')
```

```
###
Xfit = np.linspace(3, 10, 1000).reshape(-1, 1)
yfit =knn.predict(Xfit)

# Plot outputs
plt.scatter(X, y,  color='red')
plt.plot(Xfit, yfit, color='blue', linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



K Nearest neighbor (Testing Mode)

It Needs:

1. The set of stored training samples
2. Distance metric to compute distance between samples
3. The value of k, i.e., the number of nearest neighbors to retrieve

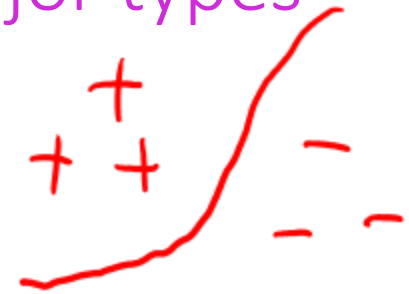
Training Mode:

- (Naïve) version: DO NOTHING !!!!

Testing Model: To classify unknown sample:

- Step1: Compute distance to all training records
- Step2: Identify k nearest neighbors
- Step3: Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

We can divide the large variety of supervised classifiers into roughly three major types



1. Discriminative

   directly estimate a decision rule/boundary

   e.g., support vector machine, decision tree,

   e.g. logistic regression, neural networks (NN), deep NN

$p(y|x)$

2. Generative:

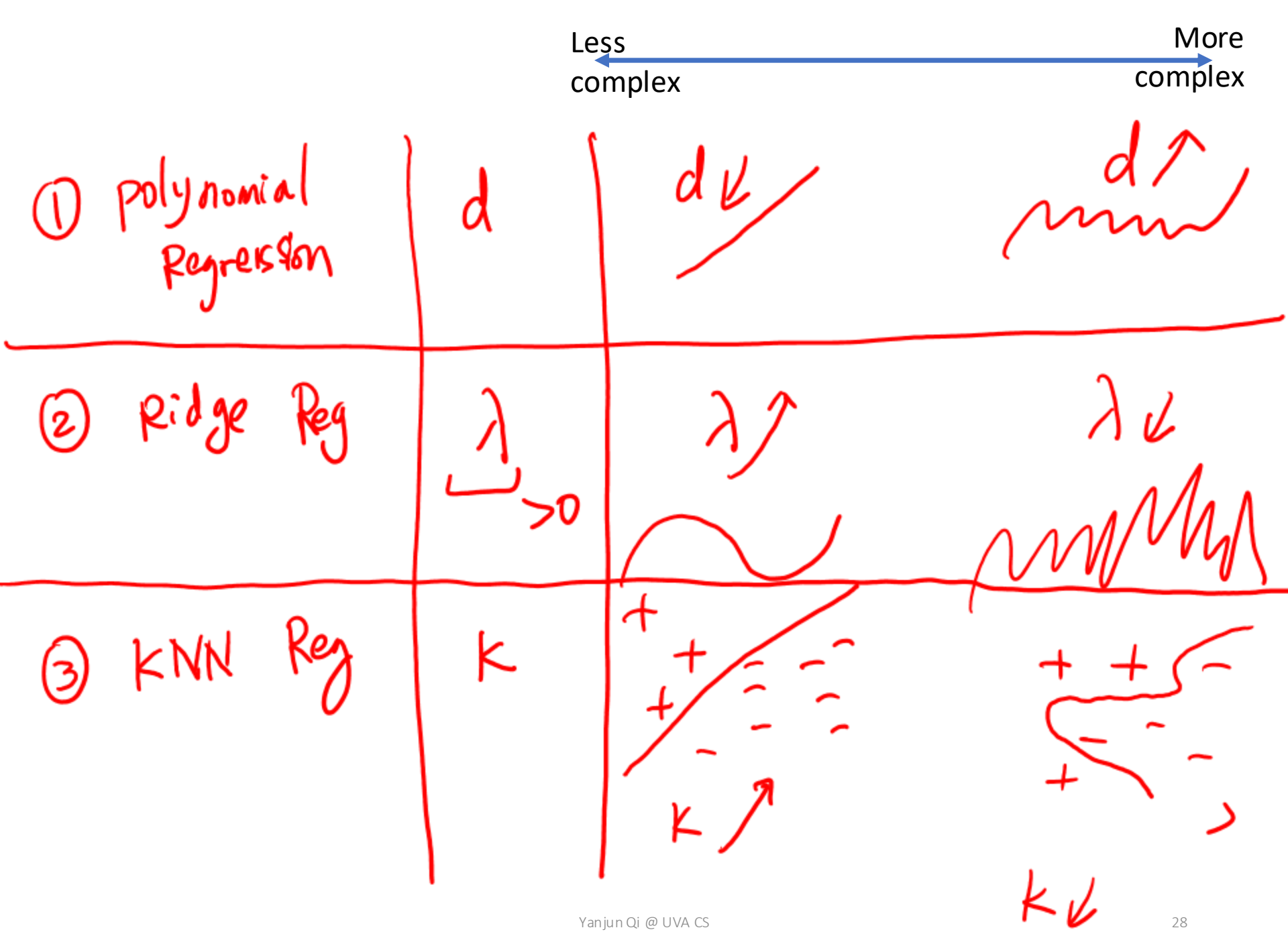
   build a generative statistical model

   e.g., Bayesian networks, Naïve Bayes classifier

$p(x|y=c)$

3. Instance based classifiers

   - Use observation directly (no models)
   - e.g. K nearest neighbors


Less complex
More complex
① polynomial Regression
d
d↙
d↗
② Ridge Reg
λ
>0
λ↗
λ↙
③ KNN Reg
k
k↗
k↙

Model Selection for Nearest neighbor classification

- Choosing the value of k:
  - If k is too small, sensitive to noise points
  - If k is too large, neighborhood may include points from other classes

- Bias and variance tradeoff
  - A small neighborhood → large variance → unreliable estimation
  - A large neighborhood → large bias → inaccurate estimation

overfit

underfit

We aim to make our trained model

- 1. Generalize Well
- 2. Computational Scalable and Efficient
- 3. Trustworthy: Robust / Interpretable
  - Especially for some domains, this is about trust!

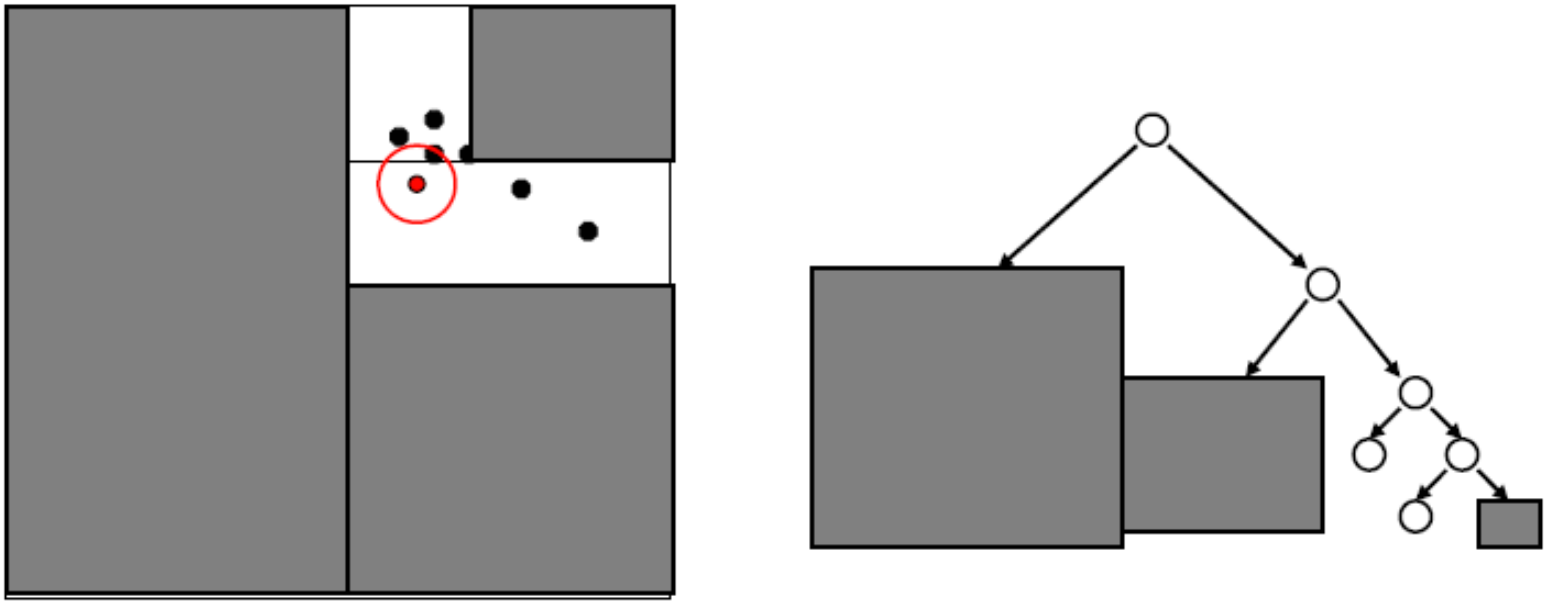
Computational Time Cost

	Train (n)	Test (m=1)
Linear Regression	$O(np^2+p^3)$	$O(p)$ $\hat{y}=\beta^T x$
KNN Reg	$O(1)$	$O(np)$+ $O(\text{sort } n\text{-}k)$???

$p = 30,000$

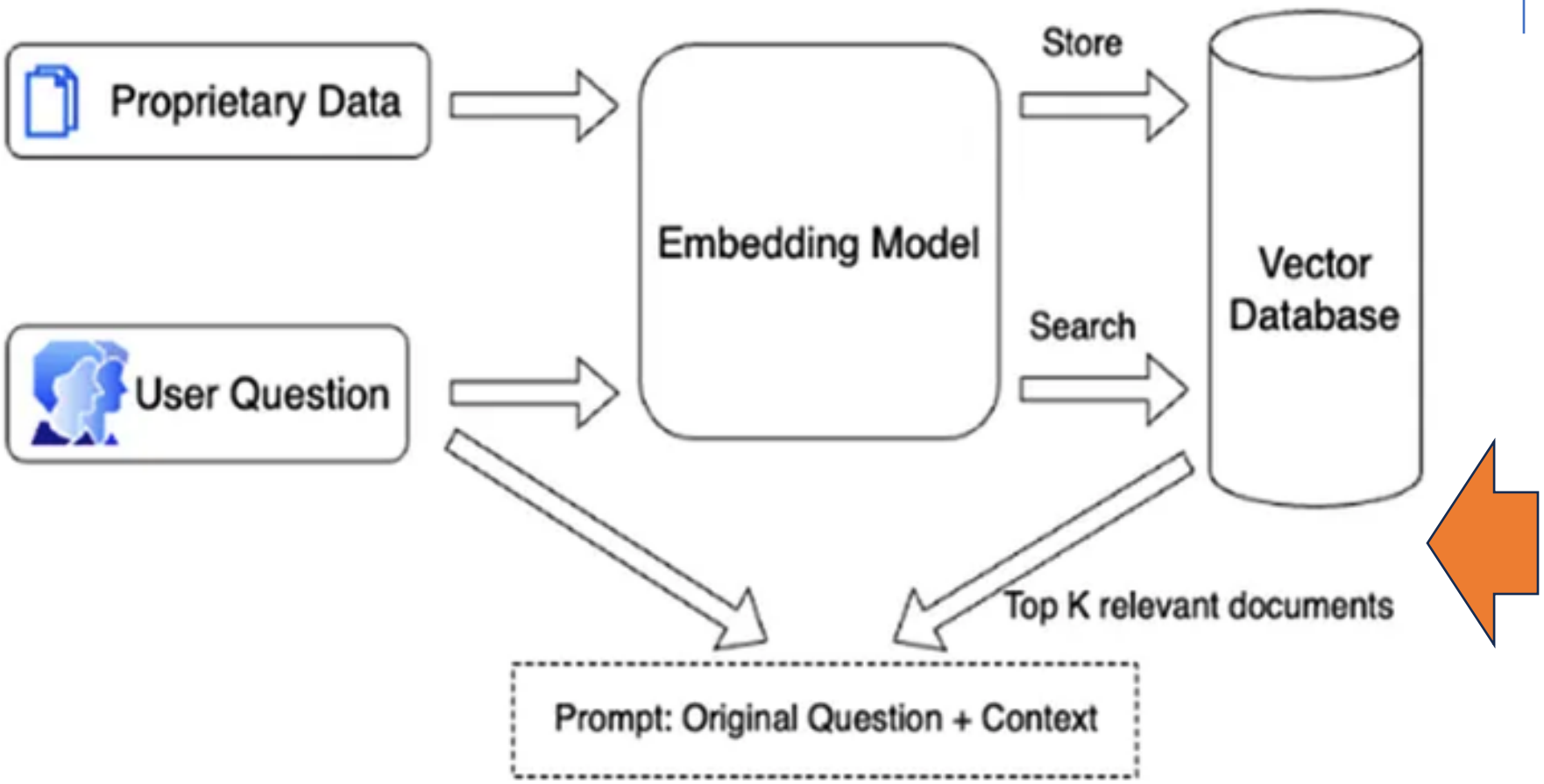
$n = 20,000$

NN Search by KD Tree



Using the distance bounds and the bounds of the data below each node, we can prune parts of the tree that could NOT include the nearest neighbor.

KNN as the Most critical component in Retrieval Augmented Generation System, e.g.:



09/30/2025 Roadmap

- TA to go over HW1
- One UVA ML club to introduce their setups and projects
- Q5
- Review Q4
- Review QA for L5-L7

10/07

10/07 /2025 Assignments

- HW2 is grading started ..
  - ➔ HW2 key walk-through next Thursday online zoom
- HW3 will get posted by tomorrow
  - Deep NN on Imaging task / Kera / mostly about learning modern DNN library
  - Programming + QA (like calculating marginal prob…)
- Next Tuesday is reading day
  - ➔ we will host makeup-Quiz Q7 next Thursday online
- Course format survey:
  - https://forms.gle/PkWGMkwHhawqf8QR8
  - Now go over the results:

Project Process

- Format:
  - Team (1~4 students)
  - Shark Tank alike Screening – https://en.wikipedia.org/wiki/Shark_Tank
  - This week: signup sheet for your team's screening sessions!
  - Next week: Initial project idea collecting!
  - TA Guangzhi will announce the process and signup sheet URL!

- Final deliverables:
  - (1) Code (Github PR to course project repo)
  - (2) Poster presentation class wide (Date: 12/09 TBD)
  - (3) Video Demo (after final exam)

10/07 /2025 Roadmp

Review L5-L8 questions

Quick Review L9-L10

Review Q5

Then Q6

quite disturbing for staying students around the right after quiz period

So we will host quiz after review / before project screening for all coming in-person sessions

Questions on L5-L8

Set 1: Bias–Variance, Overfitting, and Model Complexity

•How do bias and variance contribute to generalization error, and how do we find the "sweet spot" without knowing the true distribution?

•What are practical indicators of underfitting vs. overfitting (from graphs, learning curves, or error plots), and how do we fix each?

•How does cross-validation (choice of K) approximate generalization error, and what are the trade-offs (bias vs variance, LOOCV vs k-fold)?

•Why does zero training error often generalize poorly, and how is this linked to variance?

•Would we ever prefer high bias or high variance, and how do we reduce one while controlling the other?

Set 2: Regularization (LASSO, Ridge, Elastic Net, Generalizations)

•What are the key differences between L1 (LASSO), L2 (Ridge), and Elastic Net in terms of sparsity, robustness, computational cost, and when to use each?

•Why do L1 penalties set coefficients to zero, while L2 does not? What happens when p>np>n or when features are highly correlated?

•How does the choice of λ affect bias–variance, and how do we select it (cross-validation, validation curves)?

•Are there equivalent closed-form solutions for LASSO like Ridge has? Why are L1 and L2 chosen—what about higher-order penalties?

•When is Elastic Net preferable (e.g., grouped correlated features), and can we always default to it?

Questions on L5-L8

Set 3: k-Nearest Neighbors (kNN) and Instance-Based Learning

- How do we pick the best k (odd vs even, weighted vs unweighted, trade-offs with noisy data)?
- What is the computational cost of kNN (sorting term, memory cost), and can it overfit?
- How does the distance metric affect performance, and how are ties handled in classification?
- What are the advantages/disadvantages of kNN vs gradient descent or regularized linear models?
- In practice, how large must the dataset be to offset outliers, and is kNN more effective for regression or classification?

Set 4: Maximum Likelihood Estimation (MLE) and Probability Foundations

- Why do we usually maximize the log-likelihood instead of the likelihood itself, and how does this connect to squared error in linear regression?
- How does MLE extend from discrete distributions (e.g., coin flips) to continuous (e.g., Gaussians)?
- Why is the MLE for Bernoulli just the sample proportion, and what happens with small samples or noisy data?
- What makes MLE consistent and efficient, and are there situations where maximum likelihood may not yield the most “ideal” parameter?
- How does the bias–variance decomposition change with different loss functions (e.g., 0–1 loss, Laplace errors)?

10/07 /2025 Roadmp

Review L5-L8 questions

Quick Review L9-L10

Review Q5

Then Q6

quite disturbing for staying students around the right after quiz period

So we will host quiz after review / before project screening for all coming in-person sessions

Lecture 10: Maximum Likelihood Estimation (MLE)

➔ Probability Review

- The big picture
- Events and Event spaces
- Random variables
- Joint probability, Marginalization, conditioning, chain rule, Bayes Rule, law of total probability, etc.
- Structural properties, e.g., Independence, conditional independence
- Maximum Likelihood Estimation

If hard to directly estimate from data, most likely we can estimate

- 1. Joint probability
  - Use Chain Rule
- 2. Marginal probability
  - Use the total law of probability
- 3. Conditional probability
  - Use the Bayes Rule

$$P(A, B) = P(B)\,P(A|B)$$

$$P(B) = P(B, A) + P(B, \sim A)$$

$$= P(B, A \cup \sim A)$$

$$P(A|B)$$

$$P(B|A) = \frac{P(A, B)}{P(A)} = \frac{P(A|B)P(B)}{P(A)}$$

One Example

Assume we have a dark box with 3 red balls and 1 blue ball. That is, we have the set {r,r,r,b}. What is the probability of drawing 2 red balls in the first 2 tries?

$$P(B_1 = r, B_2 = r) = P(B_1 = r)\, P(B_2 = r \mid B_1 = r) = \frac{1}{2}$$

$$\frac{3}{4} \times \frac{2}{3}$$

$$P(B_2 = r) = P(B_1 = r, B_2 = r) + P(B_1 = b, B_2 = r)$$

$$P(B_1 = r \mid B_2 = r) = \frac{P(B_1 = r, B_2 = r)}{P(B_2 = r)}$$

MLE idea is to

- ✓ assume a particular model with unknown parameters, θ
- ✓ we can then define the probability of observing a given event conditional on a particular set of parameters. $P(Z_i|\theta)$
- ✓ We have observed a set of outcomes in the real world.
- ✓ It is then possible to choose a set of parameters which are most likely to have produced the observed results.

$$\hat{\theta} = \underset{\theta}{argmax} \; P(Z_1 \ldots Z_n|\theta)$$

Likelihood

This is maximum likelihood.

In most cases this scorer is both consistent and efficient.

$$log(L(\theta)) = \sum_{i=1}^{n} log(P(Z_i|\theta)$$

Log-Likelihood

It is often convenient to work with the Log of the likelihood function.

Deriving the Maximum Likelihood Estimate for Bernoulli

$$\log(L(p)$$

$$= \log\left[\prod_{i=1}^{n} p^{z_i}(1-p)^{1-z_i}\right]$$

$$= \sum_{i=1}^{n}(z_i \log p + (1-z_i)\log(1-p))$$

$$= \log p \sum_{i=1}^{n} z_i + \log(1-p)\sum_{i=1}^{n}(1-z_i)$$

$$= x\log p + (n-x)\log(1-p)$$

Observed data ➔ x heads-up from n trials

Deriving the Maximum Likelihood Estimate for Bernoulli

$$\arg\min_p \{-l(p)\} = \arg\min_p \{-x\log(p) - (n-x)\log(1-p)\}$$

$$\frac{dl(p)}{dp} = -\frac{x}{p} - \frac{-(n-x)}{1-p} \equiv 0$$

$$0 = -\frac{x}{p} + \frac{n-x}{1-p}$$

$$0 = \frac{-x(1-p) + p(n-x)}{p(1-p)}$$

$$0 = -x + px + pn - px$$

$$0 = -x + pn$$

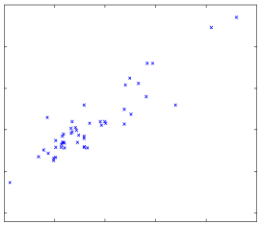
Minimize the negative log-likelihood

➔ MLE parameter estimation

$$\hat{p} = \frac{x}{n}$$

i.e. Relative frequency of a binary event

DETOUR: Probabilistic Interpretation of Linear Regression



- Let us assume that the target variable and the inputs are related by the equation:

$$y_i = \theta^T \mathbf{x}_i + \varepsilon_i$$

RV $\varepsilon \sim N(0, \sigma^2)$

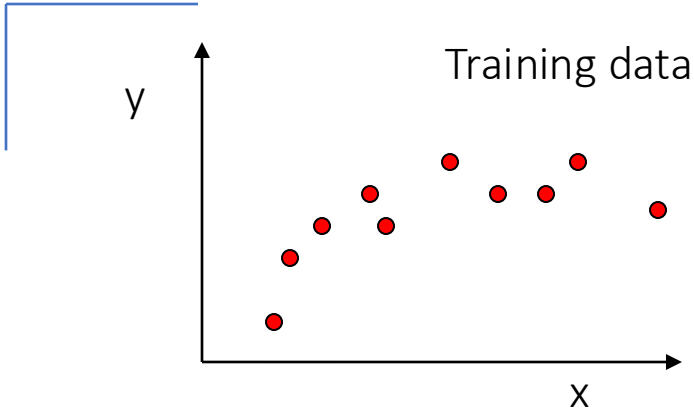
where ε is an error term of unmodeled effects or random noise

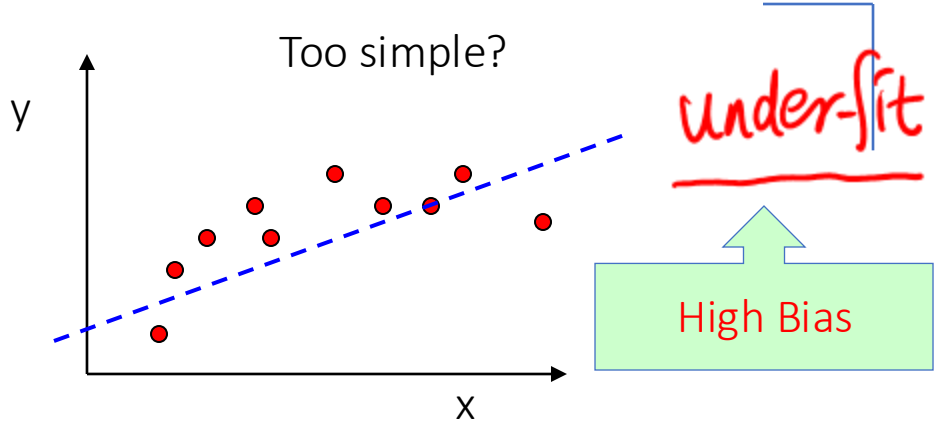
- Now assume that ε follows a Gaussian N(0,σ), then we have:

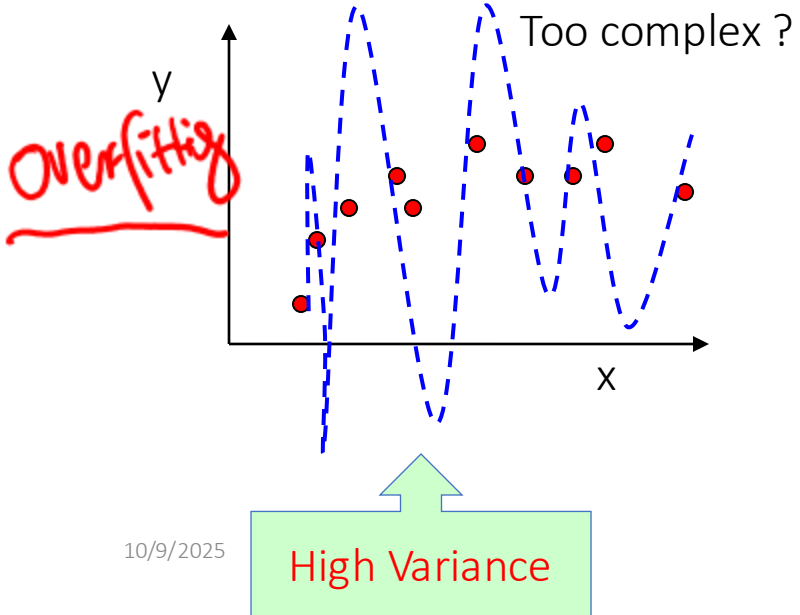
$$p(y_i \mid x_i; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y_i - \theta^T \mathbf{x}_i)^2}{2\sigma^2} \right)$$

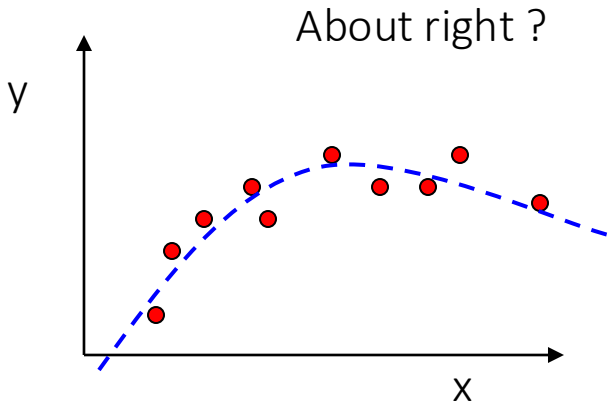
RV $y|x;\theta \sim N(\theta^T x, \sigma)$

L9: Complexity / Goodness of Fit / Generalization





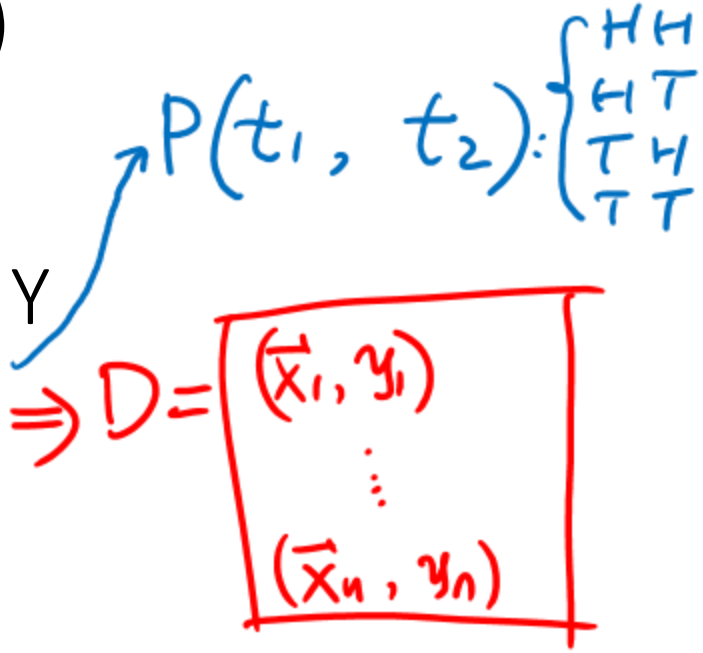




What ultimately matters: GENERALIZATION

Statistical Decision Theory (Extra)

- Random input vector: X
- Random output variable: Y
- Joint distribution: Pr(X,Y)
- Loss function L(Y, f(X))



- Expected prediction error (EPE):

$$\mathrm{EPE}(f) = \mathrm{E}(L(Y, f(X))) = \int L(y, f(x)) \Pr(dx, dy)$$

$$\text{e.g.} = \int (y - f(x))^2 \Pr(dx, dy)$$

e.g. Squared error loss (also called L2 loss)

One way to define generalization: by considering the joint population distribution

Extra

Decomposition of EPE

- When additive error model: $Y = f(X) + \epsilon,\ \epsilon \sim (0, \sigma^2)$
- Notations
  - Output random variable: Y
  - True function: f → true
  - Prediction estimator: $\hat{f}$ → D → $\hat{f}$

$$
\begin{aligned}
EPE(x) &= E[(Y - \hat{f})^2 | X = x] \\
&= E[((Y - f) + (f - \hat{f}))^2 | X = x] \\
&= E[(Y - f)^2 | X = x] + E[(f - \hat{f})^2 | X = x] \\
&= \sigma^2 + Var(\hat{f}) + \text{Bias}^2(\hat{f})
\end{aligned}
$$

Bayes Error

Irreducible / Bayes error

Bias-Variance Trade-off for EPE:

$$\text{EPE}(x) = \text{noise}^2 + \text{bias}^2 + \text{variance}$$

Unavoidable error

Error due to incorrect assumptions

Error due to variance of training samples

Slide credit: D. Hoiem

Extra, But VERY IMPORTANT

BIAS AND VARIANCE TRADE-OFF for Parameter Estimation

- θ: true value (normally unknown)
- $\hat{\theta}$: estimator
- $\bar{\theta} := E[\hat{\theta}]$ (mean, i.e. expectation of the estimator)

- Bias $E[(\bar{\theta} - \theta)^2]$
  - measures accuracy or quality of the estimator
  - low bias implies on average we will accurately estimate true parameter from training data

- Variance $E[(\hat{\theta} - \bar{\theta})^2]$
  - Measures precision or specificity of the estimator
  - Low variance implies the estimator does not change much as the training set varies

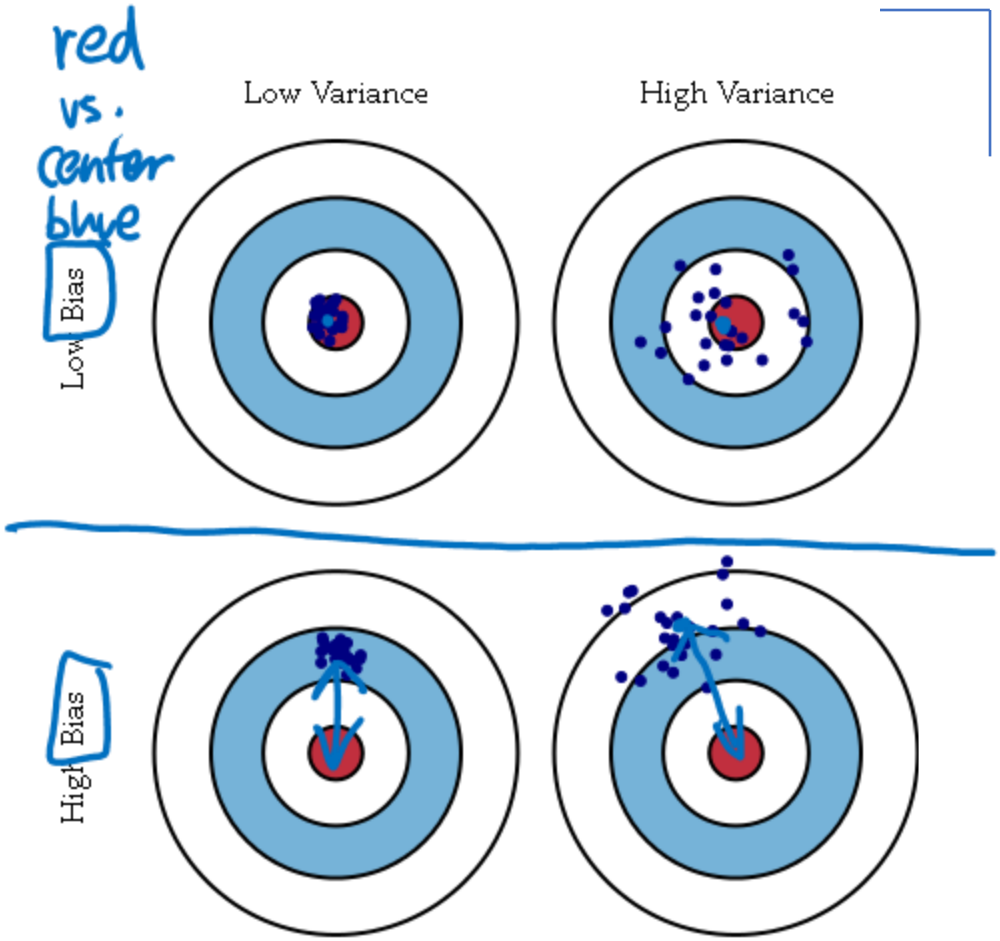
Model “bias” & Model “variance”

- Middle RED:
  - TRUE function
- Error due to bias:
  - How far off in general from the middle red

$$E[(\bar{\theta} - \theta)^2]$$

- Error due to variance:
  - How wildly the blue points spread

$$E[(\hat{\theta} - \bar{\theta})^2]$$



need to make assumptions that are able to generalize

- Underfitting: model is too “simple” to represent all the relevant characteristics
  - High bias and low variance
  - High training error and high test error

- Overfitting: model is too “complex” and fits irrelevant characteristics (noise) in the data
  - Low bias and high variance
  - Low training error and high test error

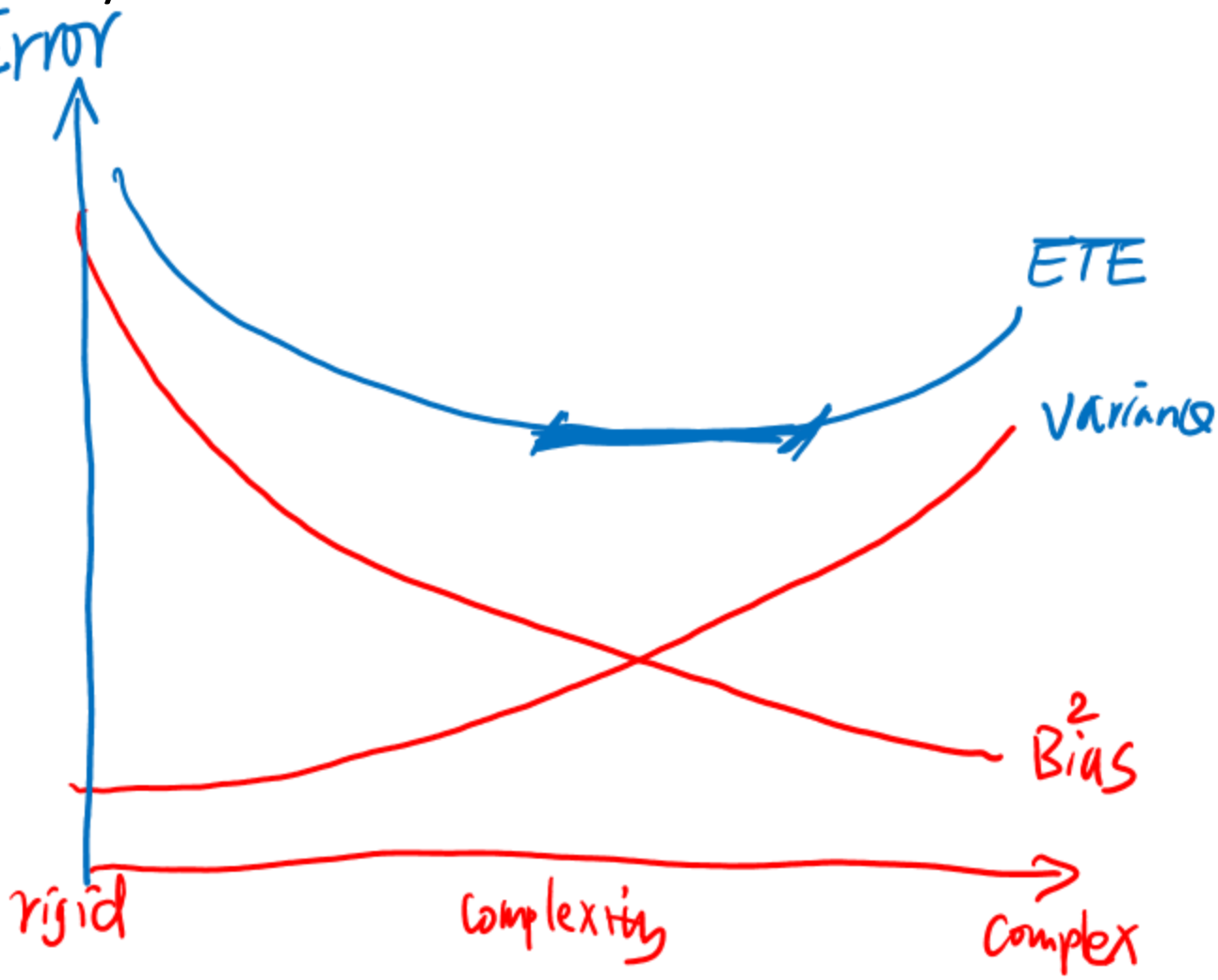
Slide credit: L. Lazebnik

Bias Variance Tradeoff

- (1) Randomness of Training Sets
- (2) Training error can always be reduced when increasing model complexity
- (3) Randomness in the Testing Error!!!
- (4) Cross Validation Error as good approximation for Expected Test error -- good appx of generalization

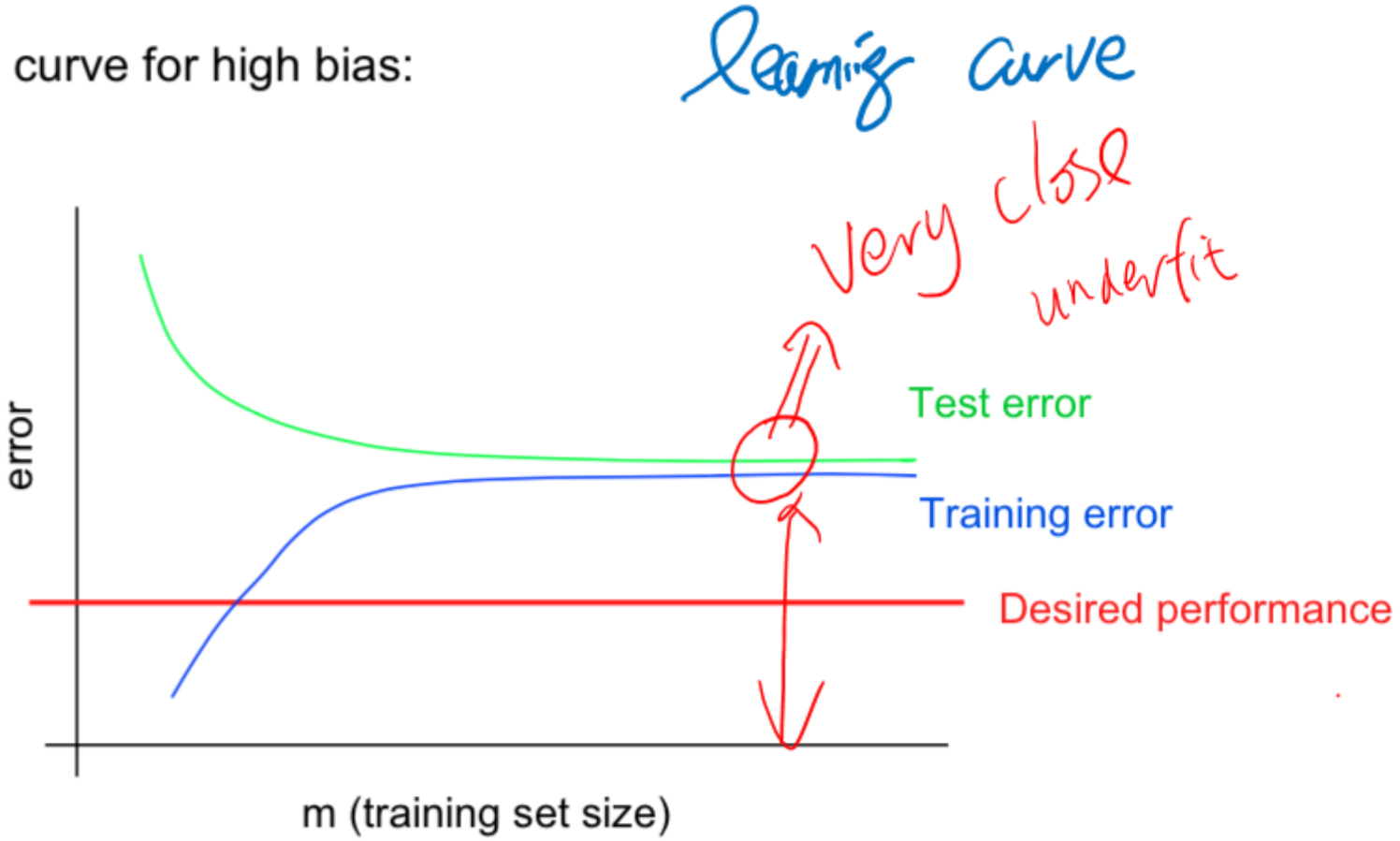
Review:
One important Control of Bias Variance Tradeoff
➔ Model Complexity

- bias decrease with model gets more complex;
- Variance increase with bigger model capacity
- Sum of Bias^2+Variance



Another important Control of Bias Variance Tradeoff ➔ Training Size (Extra)

Typical learning curve for high bias:



- Even training error is unacceptably high.
- Small gap between training and test error.

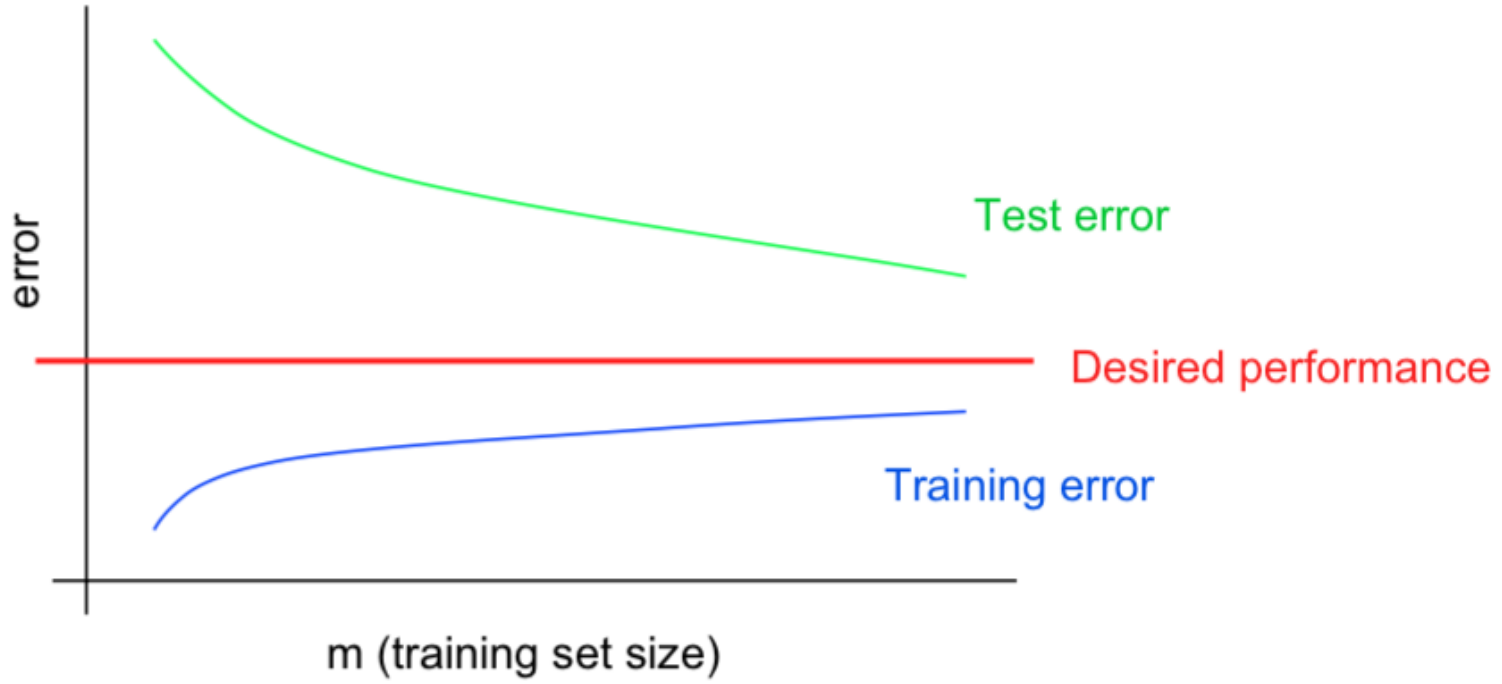
High training error and high test error

Slide credit: A. Ng

Another important Control of Bias Variance Tradeoff
➔ Training Size (Extra)

Typical learning curve for high variance:

Learning curve



Slide credit: A. Ng

How to reduce Model High Variance?

- Choose a simpler classifier
- Regularize the parameters
- Get more training data
- Try smaller set of features feature selection $n < p$
- Try feature engineering
- Try multiple models and then use all as ensemble

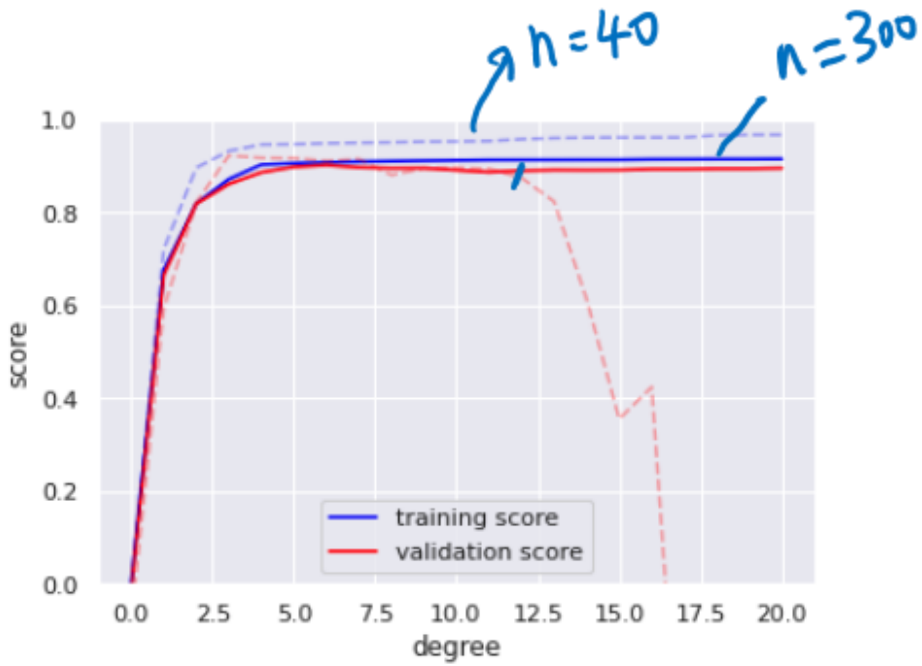
Slide credit: D. Hoiem

Take Away : Three types of plots

- (1) Sanity check (S)GD type Optimization
  - Train / Vali Loss vs. Epochs to help you
  - https://scikit-learn.org/stable/auto_examples/linear_model/plot_sgd_early_stopping.html#sphx-glr-auto-examples-linear-model-plot-sgd-early-stopping-py
- (2) Sanity check hyperparameter tuning (validation curve)
  - Train / Vali Loss vs. hyperparameter Values

    from sklearn.model_selection import validation_curve
- (3) Sanity check if your current model overfits or underfits
  - Train / Vali Loss vs. Varying Size of Training (learning curve)
  - https://scikit-learn.org/stable/auto_examples/model_selection/plot_learning_curve.html#sphx-glr-auto-examples-model-selection-plot-learning-curve-py

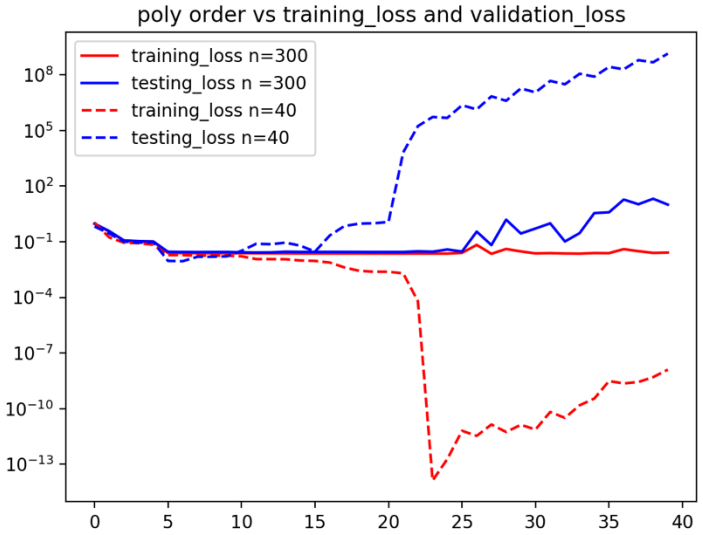
I will Code run: https://github.com/qiyanjun/2025Fall-UVA-CS-MachineLearningDeep/blob/main/notebook/L9-LearningCurves.ipynb



(1) Validation_curve_

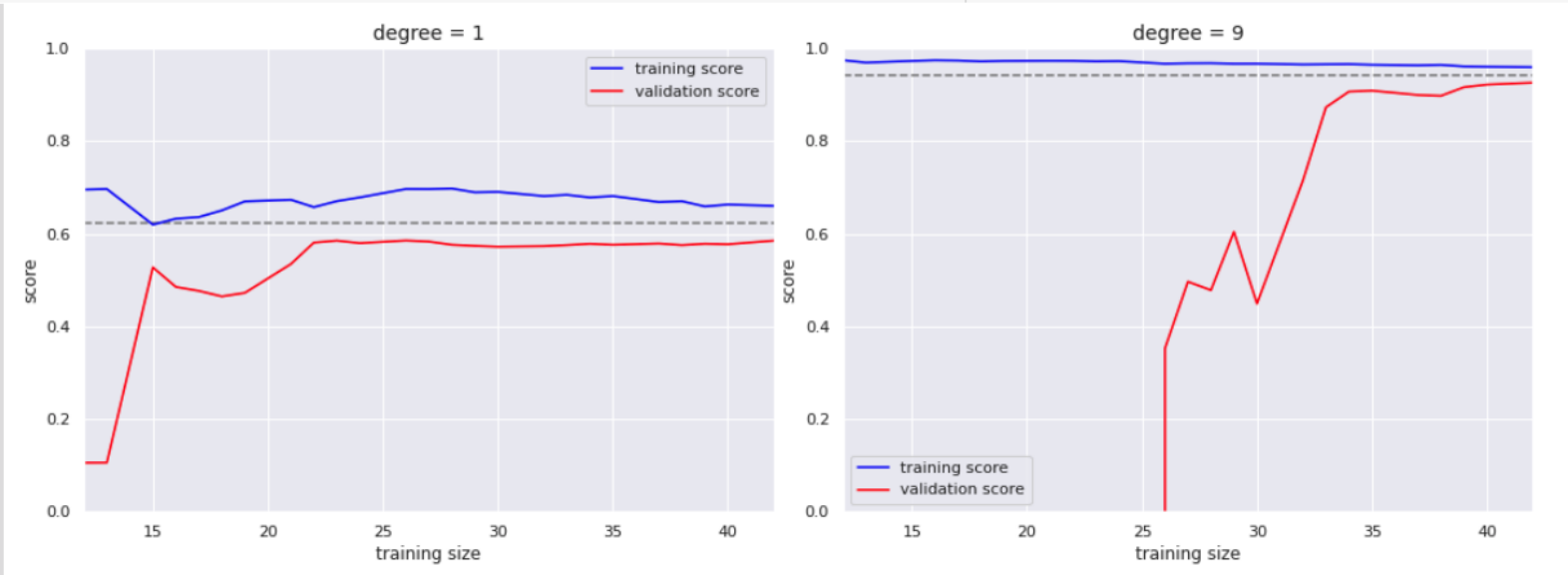
By scikitlearn Validation_curve function (normalize all metrics to positive range

https://scikit-learn.org/stable/modules/model_evaluation.html#the-scoring-parameter-defining-model-evaluation-rules

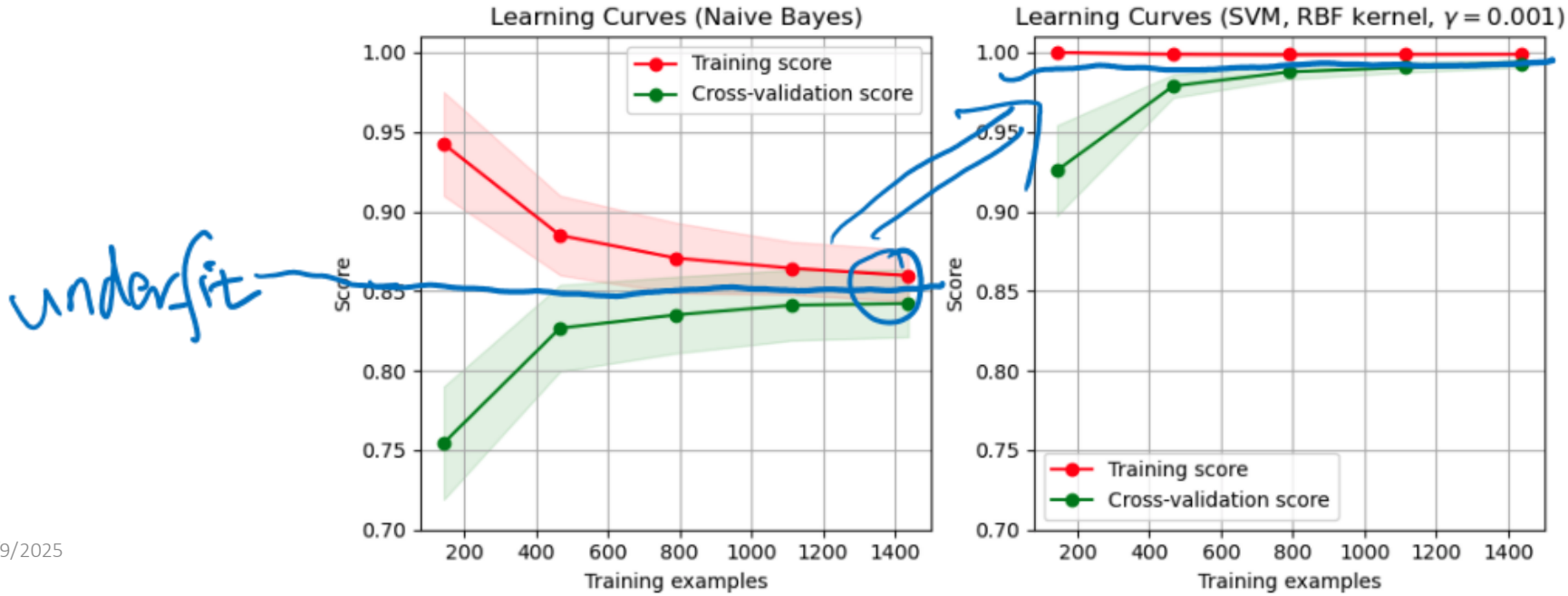


(1) Validation_curve_

By our HW2 (more close to modern deep learning library style)

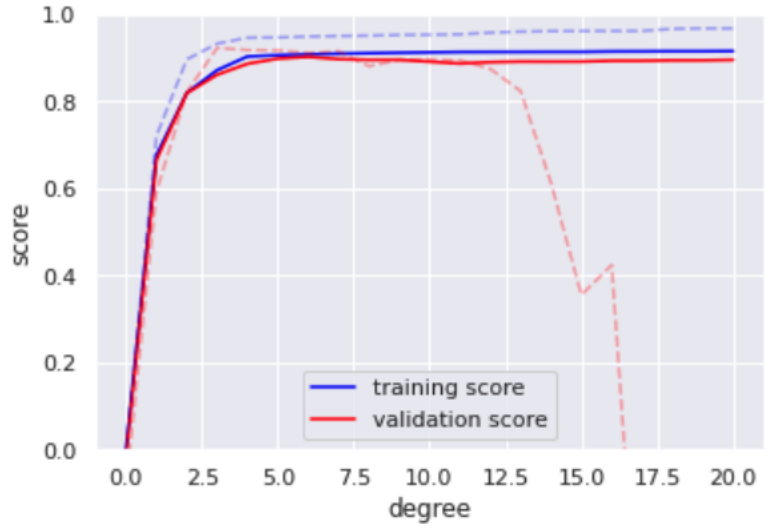


(1) Learning Curves for polynomial regression (up) and classification (down) / by scikitlearn



```
X2, y2 = make_data(200)

degree = np.arange(200)
train_score2, val_score2 = validation_curve(PolynomialR
                                            'polynomial

plt.plot(degree, np.median(train_score2, 1), color='blu
plt.plot(degree, np.median(val_score2, 1), color='red',
plt.legend(loc='lower center')
plt.ylim(0, 1)
plt.xlabel('degree')
plt.ylabel('score');
```

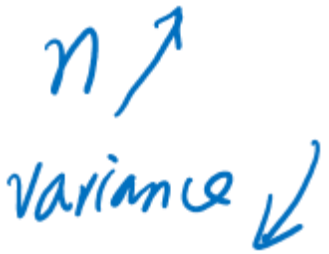

```
X2, y2 = make_data(200)
degree = np.arange(21)
train_score2, val_score2 = validation_curve(PolynomialReg
                                            'polynomialfe

plt.plot(degree, np.median(train_score2, 1), color='blue'
plt.plot(degree, np.median(val_score2, 1), color='red', l
plt.plot(degree, np.median(train_score, 1), color='blue',
plt.plot(degree, np.median(val_score, 1), color='red', al
plt.legend(loc='lower center')
plt.ylim(0, 1)
plt.xlabel('degree')
plt.ylabel('score');
```



Interesting Relation between

- the right range of model complexity
- the number of training points

Is the bias-variance trade off dependent on the number of samples? (EXTRA)



In the usual application of linear regression, your coefficient estimators are unbiased so sample size is irrelevant. But more generally, you can have bias that is a function of sample size as in the case of the variance estimator obtained from applying the population variance formula to a sample (sum of squares divided by n)…..

… the bias and variance for an estimator are generally a decreasing function of training size n. Dealing with this is a core topic in nonparametric statistics. For nonparametric methods with tuning parameters a very standard practice is to theoretically derive rates of convergence (as sample size goes to infinity) of the bias and variance as a function of the tuning parameter, and then you find the optimal (in terms of MSE) rate of convergence of the tuning parameter by balancing the rates of the bias and variance. Then you get asymptotic results of your estimator with the tuning parameter converging at that particular rate. Ideally you also provide a data-based method of choosing the tuning parameter (since simply setting the tuning parameter to some fixed function of sample size could have poor finite sample performance), and then show that the tuning parameter chosen this way attains the optimal rate.

https://www.reddit.com/r/statistics/comments/6uajyr/is_the_biasvariance_trade_off_dependent_on_the/